



Universidad
de La Laguna

SECCIÓN DE
MATEMÁTICAS

FACULTAD
DE CIENCIAS

Ricardo E. Peytaví Machado

Las matemáticas del Blackjack

Desde las leyendas del cine a la
austeridad de los números

Trabajo Fin de Grado
Grado en Matemáticas
La Laguna, Septiembre de 2018

DIRIGIDO POR
Carlos M. González Alcón

Carlos M. González Alcón
Departamento de Matemáticas,
Estadística e Investigación
Operativa
Universidad de La Laguna
38271 La Laguna, Tenerife

Agradecimientos

A Gonzalo García Pelayo por incorporarme a su equipo de jugadores profesionales.

A los profesores Nácere Hayek Calil, Luis Casasús Latorre y Pablo González Vera por lo mucho que me animaron a estudiar Matemáticas.

A los profesores de la Sección de Matemáticas de la Facultad de Ciencias por la infinita paciencia que han tenido conmigo. De forma especial, a Carlos González Alcón, tutor y amigo, cuya labor ha sido imprescindible para realizar este trabajo.

Mi infinita gratitud también a mi esposa, Rosario, que durante años me ha apoyado en cuantas aventuras he emprendido.

Resumen • Abstract

Resumen

Los juegos que ofrecen los casinos (ruleta, dados, etc.) están diseñados para que la banca tenga ventaja. El Blackjack, en cambio, escapa a esta categoría. Incorporado a las salas de juego en los años treinta del siglo pasado, comenzó a suscitar la curiosidad de los matemáticos a partir de los años sesenta, cuando intuyeron que podía existir un sistema ganador. Edward Thorp fue el primero en publicar un libro, ‘Beat the Dealer’, en el que analizaba con rigor la posibilidad de ganar en este juego. Posteriormente otros matemáticos y expertos en computación fueron afinando los sistemas. Tras realizar un somero recorrido por las experiencias de Jagger y Pelayo, los únicos que han conseguido ganar sistemáticamente en la ruleta, en este trabajo analizamos los sistemas clásicos para derrotar a la banca en el Blackjack y abordamos uno sustentado estrictamente en el cálculo de probabilidades, mediante la utilización de funciones adecuadas.

Palabras clave: *Blackjack – Recuento de cartas – Casinos– Recursividad – Cálculo de probabilidades – Detención óptima – Ruleta – Azar.*

Abstract

Games offered by casinos (roulette, craps, etc.) are designed so that the dealer has an advantage. Blackjack, however, escapes this category. Incorporated into the gambling halls in the thirties of the last century, it began to arouse the curiosity of mathematicians from the sixties, when they sensed that a winning system could exist. Edward Thorp was the first to publish a book, 'Beat the Dealer', in which he rigorously analyzed the possibility of winning in this game. Subsequently other mathematicians and computer experts were fine-tuning the systems. After a brief tour across the experiences of Jagger and Pelayo in roulette (the only ones who have managed to win in roulette), in this work we analyze the classic systems to win in Blackjack and propose one strictly supported by Probability. All necessary computer functions for this calculations have been programming.

Keywords: *Blackjack – Twenty-one – Probability – Recursion – Card counting – Twenty-one – Roulette – Casino.*

Contenido

Agradecimientos	III
Resumen/Abstract	V
Introducción	IX
1. El Blackjack	1
1.1. Algo de historia	1
1.2. Las normas del juego	1
1.3. ¿Se puede ganar al Blackjack pero no a la ruleta?	4
1.3.1. El experimento de Joseph Hobson Jagger	4
1.3.2. Gonzalo García Pelayo y el Chi-cuadrado	4
1.3.3. Edward Thorp beat the dealer	5
2. Hacia un sistema ganador	7
2.1. Formas de jugar	7
2.1.1. Libre albedrío	7
2.1.2. Estrategia básica	9
2.1.3. Estrategia básica modificada con recuento de cartas	12
2.1.4. Estrategia básica y cantidad apostada modificadas por el recuento de cartas	16
3. El cálculo directo	17
3.1. Un programa que sustituya a todos los sistemas	17
3.2. Un problema de detención óptima... y algo más	18
3.2.1. ¿Dónde parar?	18
3.3. Las probabilidades del crupier	20
3.3.1. Altas y bajas	21
3.4. La mejor decisión del jugador en cada instante	23

3.4.1. Pedir o plantarse	24
3.4.2. Doblar con 9, 10 u 11	29
3.4.3. Separar cartas	29
3.4.4. Asegurar	31
4. Funciones programadas	33
4.1. Las funciones	33
4.1.1. CrupierProb	33
4.1.2. JugadorDecide()	34
4.1.3. JugDecSep()	35
4.1.4. JugadorDobla()	36
4.1.5. JugadorAsegura()	36
5. Conclusiones y posibles ampliaciones	37
A. Anexo	39
A.1. Funciones	39
A.1.1. CrupierProb()	39
A.1.2. JugadorDecide()	42
A.1.3. JugDecSep(j, ja, n, ji, jai, sep, c, ca, sb)	44
A.1.4. JugadorDobla(j,ja,n,c,ca,sb)	47
A.1.5. JugadorAsegura(sb)	48
Bibliografía	49
Poster	51

Introducción

Los juegos que ofrecen los casinos están matemáticamente diseñados para que el cliente siempre pierda. Todos, excepto uno: el Blackjack. A diferencia de la ruleta, el Blackjack no fue concebido explícitamente mediante el cálculo de probabilidades para darle una ventaja a la banca frente al jugador, aunque desde el principio se suponía que la ‘casa’ ganaba a la larga.

En años sesenta del siglo XX, varios matemáticos del entorno de la Universidad de California empezaron a estudiar las probabilidades de vencer en el Blackjack. Dos fueron las preguntas iniciales que se formularon: ¿Es posible obtener beneficios en este juego? En caso de que sea posible, ¿cuánto se puede ganar?

La respuesta a la primera es ‘sí’: el Blackjack es un juego ventajoso para el jugador si éste toma las decisiones correctas en el transcurso de la partida. En cuanto a la segunda pregunta, los resultados fueron un tanto descorazonadores desde el principio debido a que el margen de ganancia es, en el mejor de los casos, exiguo.

Tras un somero repaso a los métodos de quienes han logrado obtener ventaja en la ruleta (por tratarse del juego más popular en cualquier casino), en este trabajo exponemos los métodos matemáticos, esencialmente de carácter estadístico, que llevaron a las primeras estrategias ganadoras en el Blackjack con los correspondientes sistemas de recuentos de naipes. Posteriormente indagamos las ventajas e inconvenientes de determinar directamente (es decir, mediante el cálculo de probabilidades) la ventaja que tiene el jugador frente al crupier en cada momento de la partida. Para ello hemos recurrido a una potente herramienta de programación, cual es la recursividad.

En el anexo final incluimos el código de las funciones que hemos programado en Matlab.

El Blackjack

1.1. Algo de historia

Se desconoce quién, o quiénes, crearon el Blackjack. Algunos autores consideran que sus orígenes están en el popular juego del *Siete y medio* que se practica, obviamente, con una baraja española.

Los partidarios de esta teoría sobre el origen del Blackjack afirman que el juego pasó de España a Francia donde se adaptó a la baraja francesa compuesta por cuatro palos (corazones, diamantes, tréboles y picas) de 13 cartas cada uno: As, 2, ..., 9, 10, J, Q, K. Las tres últimas cartas, Valet, Reina y Rey, corresponden a la Sota, Caballo y Rey de la baraja española.

Cabe suponer que el Blackjack fue exportado por los franceses a Estados Unidos. A comienzos del siglo XX se convirtió en un juego muy popular en ese país, si bien su práctica se circunscribía más al ámbito familiar que a las salas de juego. En los años treinta, cuando Las Vegas se convirtió en la capital mundial del juego, casi todos los casinos lo ofrecían a sus clientes junto a la imprescindible ruleta y los dados.

1.2. Las normas del juego

El Blackjack se practica con una baraja francesa formada por cuatro palos de trece cartas cada uno, numeradas desde el As, que vale 1 u 11 (lo que más le convenga al jugador o al crupier en sus respectivas manos), hasta el 10. Cada una de las tres figuras, J, Q y K, también vale diez.

El juego consiste en llegar lo más cerca posible a 21 sin superar esta puntuación, pues se pierde inmediatamente si ocurre esto.

El crupier juega contra uno o varios participantes, hasta un máximo de seis o siete dependiendo de las normas del casino.

Al inicio de la partida el crupier puede utilizar una única baraja de 52 naipes, o bien un mazo de dos, cuatro, seis u ocho barajas. Con una o dos barajas sostiene las cartas con una mano y las reparte con la otra. Cuando emplea cuatro, seis u ocho barajas extrae las cartas de una caja denominada sabot. En los casinos americanos hay mesas de Blackjack de una, dos, cuatro y seis barajas, aunque en algunos ofrecen un juego de ocho. En Europa lo habitual es jugar con seis barajas.

Cuando comienza una mano, el crupier le da una carta a cada uno de los jugadores de la mesa y toma una carta para él. Todas las cartas se depositan sobre el tapete hacia arriba para que se pueda ver su valor. A continuación, el crupier reparte una segunda carta a cada jugador. En este punto se produce una diferencia importante entre los casinos norteamericanos y los europeos (en general, con los de todo el mundo), pues en Estados Unidos el crupier toma una segunda carta boca abajo (sin que se conozca su valor) tras repartir el segundo naipe a los jugadores. Esta norma aumenta sensiblemente la ventaja del jugador frente a la banca.

Lo normal, empero, es que el crupier no tome su segunda carta hasta que todos los participantes hayan jugado sus manos. Esquemáticamente, el juego se desarrolla así:

- 1) Cada jugador recibe una primera carta que el crupier deposita en su casilla hacia arriba. El crupier se da una carta a sí mismo.
- 2) Cada jugador recibe una segunda carta también hacia arriba.
- 3) Comienza la ronda de decisiones. Si el primer jugador no tiene 21 o blackjack (un blackjack es un 21 conseguido sólo con dos cartas: un as y una carta de valor diez), el crupier le pregunta si se planta o pide.
 - a) Si el jugador se planta, la apuesta que ha hecho y las cartas que ha recibido se quedan en su casilla y el crupier pasa al jugador siguiente.
 - b) Si el jugador pide, el crupier le da una carta. Si con este naipe el jugador todavía no se ha pasado ni ha conseguido 21 puntos, el crupier le pregunta si quiere una carta más. El proceso concluye cuando el jugador se pasa o decide plantarse.
 - c) Si el jugador se pasa, pierde de forma inmediata. El crupier le cobra la apuesta y le retira las cartas.
- 4) Cuando todos los jugadores han completado sus manos, el crupier juega la suya. En realidad, el crupier no decide nada, ni siquiera en el momento de jugar su propia mano, pues está obligado a pedir mientras tenga menos de 17 puntos y a plantarse con 17 o más. Existen dos posibilidades:
 - a) Si el crupier se pasa ganan todos los jugadores que siguen ‘vivos’, es decir, aquellos que no se han pasado, sea cual sea su puntuación.
 - b) Si el crupier obtiene entre 17 y 21 puntos, o un blackjack, ganan los jugadores que tienen más puntos que el crupier, pierden los que tienen menos y empatan los que han obtenido los mismos.

- 5) El balance económico del juego es el siguiente:
 - a) Los jugadores con más puntos que el crupier ganan una cantidad igual a la apostada (1:1).
 - b) Los que tienen menos puntos, pierden la apuesta. (-1:1).
 - c) Los que igualan en puntos al crupier, no reciben premio pero no pierden la apuesta (0:1).
 - d) Los jugadores que han conseguido un blackjack (21 con dos cartas) reciben como premio 1,5 veces la apuesta (3:2)
- 6) El jugador realiza la apuesta antes de comenzar cada mano y no puede retirarla hasta que concluya ésta. Sí puede, en cambio, hacer una apuesta adicional en los siguientes casos:
 - a) **Separar cartas.** Si las dos primeras cartas que recibe son del mismo valor (no es necesario que sean iguales, sino que representen los mismos puntos. Verbigracia, un cinco de diamantes es equivalente a un cinco de tréboles y un diez de corazones a un rey de picas) puede separarlas y jugar dos manos independientes. Para separar debe depositar una apuesta igual a la cantidad inicial. Si se pasa en la primera de las manos separadas, o decide plantarse, sigue jugando con la otra hasta que se pase o se plante. Cuando el crupier concluye su mano, le paga, le cobra o le deja una u ambas apuestas según el resultado que hayan obtenido él y el jugador. En España (y normalmente en toda Europa) lo habitual es que al jugador se le permita reabrir indefinidamente las manos si al separar las dos primeras cartas recibe una del mismo valor. Los casinos de Las Vegas (y de Estados Unidos en general) suelen prohibir la reapertura de cartas o, en el mejor de los casos, la restringen a un máximo de tres separaciones. Esta norma tiene poco sentido en el Blackjack de un mazo ya que al sólo haber cuatro cartas del mismo valor (salvo en el casos de los dieces, en el que hay dieciséis: cuatro dieces propiamente dichos más las doce figuras que también valen diez) la probabilidad de recibir sucesivas cartas con idéntica puntuación es pequeña.
La posibilidad de reabrir cartas aumenta la ventaja del jugador.
 - b) **Doblar la apuesta.** Si con las dos primeras cartas el jugador suma 9, 10 u 11 puntos puede duplicar la apuesta. Es lo que se conoce como doblar. Existe una penalización: si el jugador dobla, solo recibe una carta más. Esto implica que se puede doblar con 9, obtener un 2 y tener que plantarse obligatoriamente con 11, lo cual es muy desventajoso pues tanto un 10 como un 11 (lo veremos posteriormente) suponen una puntuación inmejorable para acceder a 21.
En Estados Unidos se permite doblar con cualquier puntuación siempre que el jugador sólo haya recibido las dos primeras cartas. En contrapartida, no se permite doblar después de haber separado, mientras que en Europa sí es posible esta última opción.

- c) **Seguro.** Si el crupier tiene un As como primera carta, el jugador puede asegurar su apuesta sea cual sea su puntuación. En realidad se trata de una apuesta adicional cuyo importe máximo sólo puede ser la mitad la apuesta inicial. Si el crupier obtiene un blackjack (un 21 sólo con dos cartas), el jugador gana el seguro y cobra el doble de la cantidad realizada en esta apuesta (2:1). Con cualquier otra puntuación del crupier, el jugador pierde el seguro pero no, necesariamente, la apuesta inicial ya que esta se desarrolla de forma independiente.

La correcta gestión del seguro es muy importante para que el jugador tenga ventaja. Tanto es así, que algunos sofisticados sistemas de recuento de cartas llevan una cuenta paralela cuyo único fin es determinar si, de acuerdo con las cartas que faltan por salir, es rentable o no ‘comprar’ el seguro.

1.3. ¿Se puede ganar al Blackjack pero no a la ruleta?

1.3.1. El experimento de Joseph Hobson Jagger

A finales del siglo XIX, el ingeniero inglés Joseph Hobson Jagger se preguntó si las ruletas de los casinos mostrarían tendencias debido a defectos de fabricación y desgaste de sus elementos mecánicos. Reunió a un grupo de colaboradores para que anotasen los números que salían en cada ruleta del casino de Montecarlo y y llegó a la conclusión de que casi todas las ruletas estaban lo suficientemente bien equilibradas para que no se produjesen desviaciones significativas, salvo una en la que varios números salían con una frecuencia mayor a la que les correspondía. Jagger apostó a esos números y acumuló una pequeña fortuna.

1.3.2. Gonzalo García Pelayo y el Chi-cuadrado

Un siglo después de la exitosa aventura de Jagger, Gonzalo García Pelayo, director de cine y productor musical español, tuvo la misma idea sin haber oído nada sobre el ingeniero inglés y su sistema. Sólo con conocimientos elementales de matemáticas, García Pelayo diseñó un estudio estadístico que, según comprobaron posteriormente algunos matemáticos, era un remedo muy práctico y eficaz del Chi-cuadrado.

Salvo estas excepciones, cualquier intento de buscar un sistema matemático capaz de derrotar a la ruleta supone un esfuerzo inútil que, inevitablemente, conduce a la melancolía. Las ‘martingalas’ de nada sirven contra este juego.

En la ruleta el jugador tiene un 2,7% en contra si juega a números y el 1,3%, también en su contra, si opta por las suertes sencillas. Lo suficiente para arruinar las fortunas mejor dotadas. No obstante, lo que le ‘cobran’ los

casinos a sus clientes por jugar es bastante menos que la cuota pagada en la lotería nacional y otras apuestas del Estado. Un detalle que no debemos perder de vista.

No sabemos si Jagger apostó a números o a suertes sencillas cuando acudió a Montecarlo. Nos consta, en cambio, que el equipo de García Pelayo siempre apostó a números. Esto supone que se enfrentaba permanentemente a un listón situado en el peor de los porcentajes negativos. Por eso no les servían las ruletas ligeramente desequilibradas, sino aquellas en las que el sesgo fuese importante.

1.3.3. Edward Thorp beat the dealer

Cuando pasamos de la ruleta al Blackjack encontramos un panorama completamente distinto. La bola de la ruleta no tiene memoria. Lo mismo ocurre si en vez de 37 números tenemos los 52 naipes de una baraja francesa si sacamos una carta cada vez y luego la devolvemos al mazo y lo barajamos convenientemente antes de proceder a una nueva extracción. En ambos casos estamos ante una elección con reemplazamiento (el reemplazamiento al lanzar una bola en la ruleta o un par de dados sobre el tapete verde es implícito). Es decir, tratamos con sucesos aleatoriamente independientes. La situación cambia radicalmente si un naipe no es devuelto al mazo una vez mostrado, sino colocado en una ‘pila de descarte’ (una denominación elegida a propósito por una razón que veremos pronto). Si ha salido el ‘2 de corazones’, está claro que ya no volverá a salir hasta que agotemos el mazo entero, tomemos las cartas de la pila en la que las hemos ido depositando, las mezclamos de forma adecuada¹ y reanudemos el proceso de extracción. Varios matemáticos del entorno de la Universidad de California se plantearon la pregunta de si se podía ganar el Blackjack teniendo en cuenta que probabilidad de que tanto el jugador como el crupier tengan una determinada puntuación en cada mano depende de los naipes aparecidos previamente y depositados en la pila de descarte. Uno de estos matemáticos fue Edward O. Thorp. Thorp utilizó un ordenador IBM 704² para demostrar, mediante miles de simulaciones, lo que ya sospechaban los jugadores de Blackjack: la abundancia de cartas bajas aún por salir favorecen al crupier y la de cartas altas al jugador. Esto equivale a afirmar que si han salido muchas cartas altas y se han depositado en la pila de descarte, la desventaja del jugador es mayor y viceversa.

¹ Hay estudios realizados con criterios matemáticos sobre la forma de barajar un mazo de cartas para que su nuevo orden sea aceptablemente aleatorio en relación con el anterior; el asunto no es tan sencillo.

² El IBM 704 fue el primer ordenador producido en serie por esta compañía con un hardware sustentado en la aritmética de coma flotante.

Hacia un sistema ganador

2.1. Formas de jugar

Hay cuatro niveles de juego en el Blackjack que van desde el que practican los clientes ocasionales del casino hasta el desarrollado por los apostadores profesionales.

2.1.1. Libre albedrío

En este nivel, para pedir cartas, plantarse, separar, doblar o asegurar sus apuestas el jugador no utiliza ningún criterio procedente de ensayos estadísticos ni del cálculo de probabilidades. Con independencia de la carta que muestre el crupier, pide siempre que tiene 11 puntos o menos y suele plantarse a partir de 12, que es cuando empieza a correr el riesgo de pasarse. No obstante, también suele pedir con más de doce si así se lo dicta una corazonada o se lo aconseja el amigo que está apostando junto a él.

Esta forma intuitiva de proceder convierte al Blackjack en uno de los juegos más rentables para el casino. Veamos un ejemplo muy significativo, cual es el de los clientes que deciden no arriesgarse jamás a pasarse.

Hemos visto que el crupier se limita a poner cartas sobre el tapete. Le da a cada jugador los naipes que le pide hasta que se pasa o decide plantarse. Sin pérdida de generalidad, supongamos que el crupier se enfrenta a un sólo jugador. Cuando dicho jugador ‘está servido’, el crupier se da cartas a sí mismo mientras tiene 16 puntos o menos y se planta con 17 o más, sin considerar cuál es la puntuación del jugador.

Podría darse el caso de que el jugador tuviese 19 puntos y el crupier 17, por ejemplo. En tal situación, y a la vista de que un 19 le gana a un 17, el crupier podría intentar pedir una carta adicional con la esperanza de que le cayese un 3 o un 4, con lo cual le ganaría al apostador al conseguir 20 o 21

puntos respectivamente, o al menos empatar con un 2 que lo situase en 19. No obstante, las normas, lo reiteramos, le impiden pedir cuando ha llegado a 17 o más.

Un ejemplo

En el desarrollo de este TFG hemos programado una función en Matlab que calcula las probabilidades del crupier de obtener entre 17 y 21 puntos, o pasarse, a partir de una puntuación inicial. Dicha función, denominada **CrupierProb**, utiliza como variables c (puntuación inicial del crupier), ca (presencia de un As con valor 11), nc (número de cartas utilizadas por el crupier) y sb (cartas que aún no han salido).

En un capítulo posterior describiremos con detalle como realiza los cálculos la función **CrupierProb**. De momento nos limitamos a usarla para ver qué le ocurre a un jugador si decide no arriesgarse nunca a pasarse.

Puesto que el crupier no va a detenerse antes de 17, el jugador sólo puede ganar si se planta con menos de esa puntuación si el crupier se pasa. Esto vale no únicamente para un 12 del jugador (en adelante utilizaremos la notación $j = x$ para indicar que el jugador tiene x puntos y $c = x$ para denotar que el crupier tiene x puntos), sino también para $j = \{13, 14, 15, 16\}$. A efectos prácticos, un jugador tiene la misma probabilidad de ganar plantándose con un 12 que con un 16, aunque la probabilidad de pasarse (y perder de inmediato), obviamente, es mucho mayor si pide con 16. También veremos esto con detalle posteriormente.

La siguiente tabla muestra las probabilidades que tiene el crupier de pasarse de acuerdo con su primera carta, suponiendo que cuando inicia su jugada tenga una baraja de un mazo que esté intacta; es decir, que tenga las 13 cartas de cada palo con un total de 52.

As	2	3	4	5	6	7	8	9	10
0,1180	0,3523	0,3743	0,3932	0,4172	0,4217	0,2645	0,2458	0,2312	0,2141

Tabla 2.1. Probabilidades de que el crupier se pase respecto a su primera carta.

Subrayamos que si el crupier tiene un As, el jugador pierde el 88,2 % de las veces ($1 - 0,1180 = 0,882$) si decide plantarse sin haber llegado como mínimo a 17 puntos. La tabla 2.1 también nos muestra otro aspecto interesante que adelantamos desde ahora: la probabilidad de que el crupier se pase aumenta a medida que tiene más puntos con sus primera carta, pero sólo hasta seis. A partir de ahí descende bruscamente y continúa bajando hasta que tiene diez.

Esto responde a la forma de jugar del crupier. Con un 5 o un 6 es fácil que llegue a 15 o 16 y entonces está próximo a pasarse. Con un 7 o más como primera carta, en cambio, le resulta más fácil llegar a una puntuación entre 17 y 21, con lo que se detiene antes de pasarse.

En definitiva, el jugador tiene las mismas probabilidades de ganar tanto si se planta con un 12 como si lo hace con un 13, un 14, 15 o 16, pues está apostando a que el crupier se pasará. Sin embargo, no tiene las mismas probabilidades de perder si pide con un 12 que si lo hace con un 16. Teniendo en cuenta la carta del crupier, ¿hasta dónde debe arriesgarse a pedir?

Aquí entramos en un caso típico del Problema de la detención óptima. “Toda decisión entraña un riesgo”, señala Ted Hid¹ en ‘El problema de la detención óptima’. “Elegir el momento adecuado para continuar con una acción puede resultar crucial”.

Hemos programado unas funciones en Matlab capaces de determinar, mediante un cálculo de probabilidades llevado hasta sus últimas consecuencias, lo que le conviene hacer a un jugador de Blackjack en cada momento de acuerdo con su puntuación, la carta del crupier y las cartas que quedan en el sabot. Este programa no se puede usar en un casino porque en las salas de juego están prohibidos todos los dispositivos que ayuden a tener ventaja en cualquier juego.

Los matemáticos que abordaron el problema de ganar en el Blackjack necesitaban un sistema que pudieran usar los jugadores sin ayuda de mecanismos externos. Un sistema que no requiriese más cálculos que los realizables por un cerebro normal en condiciones normales.

2.1.2. Estrategia básica

El primer paso hacia la consecución de un sistema ganador fue la *estrategia básica*, establecida por Thorp en ‘Beat the Dealer’. Se trata de un conjunto de reglas sobre lo que debe hacer el jugador dependiendo, solamente, de los puntos que tenga en cada momento y de la carta del crupier. El jugador las memoriza (se presentan en forma de tablas) y las aplica durante la partida.

Las tablas de estrategia básica varían de acuerdo con las normas del Blackjack que ofrece cada casino. En el caso europeo, con un sabot de seis mazos de naipes, posibilidad de doblar sólo si las dos primeras cartas suman 9, 10 u 11 puntos, opción de doblar después de separar pares y la posibilidad de volver a separar pares. La tabla 2.2 muestra dicha estrategia básica.

El jugador debe memorizar esta estrategia y aplicarla en cada caso. Por ejemplo, frente a un 2 del crupier debe pedir si tiene 12 pero plantarse si tiene 13. Esta es la expresión más simple, en cualquier caso, del Problema de la detención óptima aplicado al Blackjack.

¹ Theodore P. Hill es doctor en Matemáticas por la Universidad de California y profesor emérito del Instituto de Tecnología de Georgia.

	2	3	4	5	6	7	8	9	T	A
4	H	H	H	H	H	H	H	H	H	H
5	H	H	H	H	H	H	H	H	H	H
6	H	H	H	H	H	H	H	H	H	H
7	H	H	H	H	H	H	H	H	H	H
8	H	H	H	H	H	H	H	H	H	H
9	H	D	D	D	D	H	H	H	H	H
10	D	D	D	D	D	D	D	D	H	H
11	D	D	D	D	D	D	D	D	H	H
12	H	H	S	S	S	H	H	H	H	H
13	S	S	S	S	S	H	H	H	H	H
14	S	S	S	S	S	H	H	H	H	H
15	S	S	S	S	S	H	H	H	H	H
16	S	S	S	S	S	H	H	H	H	H
17	S	S	S	S	S	S	S	S	S	S
18	S	S	S	S	S	S	S	S	S	S
19	S	S	S	S	S	S	S	S	S	S
20	S	S	S	S	S	S	S	S	S	S

Tabla 2.2. Tabla de estrategia básica para seis mazos y normas europeas. La línea superior indica los puntos el crupier y la columna de la izquierda los puntos del jugador. H (hit): pedir. S (stand): plantarse. D (double): doblar la apuesta.

Existe la correspondiente estrategia básica para jugar las manos blandas (aquellas en las que interviene un ‘as’ con valor 11) que recogemos en la [tabla 2.3](#).

	2	3	4	5	6	7	8	9	T	A
A,2	H	H	H	H	H	H	H	H	H	H
A,3	H	H	H	H	H	H	H	H	H	H
A,4	H	H	H	H	H	H	H	H	H	H
A,5	H	H	H	H	H	H	H	H	H	H
A,6	H	H	H	H	H	H	H	H	H	H
A,7	S	S	S	S	S	S	S	H	H	H
A,8	S	S	S	S	S	S	S	S	S	S
A,9	S	S	S	S	S	S	S	S	S	S

Tabla 2.3. Tabla de estrategia básica para manos blandas. H (hit): pedir. S (stand): plantarse.

Cuando las dos primeras cartas que recibe el jugador son del mismo valor (no necesariamente iguales, sino del mismo valor: un rey de corazones es

equivalente a una reina de diamantes o a un diez de picas), puede separarlas si lo considera oportuno. La tabla 2.4 recoge la estrategia básica para tomar esta decisión de la forma más ventajosa.

	2	3	4	5	6	7	8	9	T	A
A,2	H	H	H	H	H	H	H	H	H	H
A,A	P	P	P	P	P	P	P	P	P	H
2,2	P	P	P	P	P	P	H	H	H	H
3,3	P	P	P	P	P	P	H	H	H	H
4,4	H	H	H	P	P	H	H	H	H	H
5,5	D	D	D	D	D	D	D	D	H	H
6,6	P	P	P	P	P	H	H	H	H	H
7,7	P	P	P	P	P	P	H	H	H	H
8,8	P	P	P	P	P	P	P	P	H	H
9,9	P	P	P	P	P	S	P	P	S	S
T,T	S	S	S	S	S	S	S	S	S	S

Tabla 2.4. Estrategia básica para separar. H (hit): pedir. S (stand): plantarse. P (pair): separar. D (double): doblar.

Aunque parezca una tarea ardua, no resulta difícil memorizar estas tablas. En cualquier caso, esta es la parte más sencilla de cualquier sistema ganador en el Blackjack.

La estrategia básica no es suficiente para ganar. En un juego de un mazo con las normas de los casinos situados en el Strip de Las Vegas (los más favorables para los aficionados al juego del 21), la estrategia básica consigue que el cliente esté igualado con la banca. Con las reglas de los casinos europeos (la más importante de ellas es tener que jugar con un sabot de seis barajas), la desventaja para el jugador, calculada mediante repetidas simulaciones, es de un $-0,55\%$ aproximadamente.

Últimamente los casinos han introducido máquinas de barajar para el Blackjack. Los naipes no son depositados en una pila de descarte cuando concluye cada mano hasta que se acaba el sabot, sino que son reintroducidos en una máquina mezcladora con lo cual pueden volver a salir de forma inmediata. Esto supone jugar con un sabot cuyo número de cartas es infinito o, equivalente, a quitarle al Blackjack la condición de juego con probabilidad condicionada. La desventaja del jugador de estrategia básica sube hasta el $-0,87\%$. No obstante, resulta menos negativa que la existente en la ruleta, que es del $-2,7\%$ en apuestas a números y del $-1,3\%$ en suertes sencillas.

En este trabajo describimos los métodos utilizados para calcular con criterios matemáticos estas tablas de estrategia básica. Asimismo, hemos programado varias funciones que permitirían prescindir de ellas y, aun así, incrementar la ven-

taja del jugador. Esto lo veremos en el siguiente capítulo. Antes damos un paso más.

2.1.3. Estrategia básica modificada con recuento de cartas

Si observamos la tabla 2.2 vemos que algunas decisiones están en el límite de pedir o plantarse. Por ejemplo, si el jugador tiene 12 puntos frente a un 2 del crupier debe pedir pero con 13 puntos lo recomendable es plantarse. Lo mismo sucede (lo veremos con detalle en su momento) con la decisión de 16 puntos cuando el crupier tiene un 10. La función `JugadorDecide()`, que hemos programado en Matlab (y que también describiremos en su momento) indica la conveniencia de pedir o plantarse dependiendo de que en el sabot el número e dieces (de cartas altas, en general) esté por encima o por debajo del correspondiente a una (o a varias, según sea el caso) baraja completa. Es decir, el recuento de cartas modifica una decisión de la estrategia básica.

Vayamos al principio.

Una vez establecida la estrategia básica, los matemáticos que estudiaban la posibilidad de ganar en el Blackjack analizaron el efecto de extraer una o varias cartas de la baraja sobre la ventaja del jugador. De acuerdo con las reglas del juego, cabía pensar a priori que las cartas bajas (2, 3, 4, 5 o 6) favorecen al crupier porque lo ayudan a sumar puntos hasta situarse entre 17 y 21 sin pasarse y, a la vez, perjudican al jugador por dos motivos:

1. Lo dejan muy corto cuando dobla con 9, 10 u 11 (recordemos que al doblar sólo tiene derecho a recibir una única carta).
2. Lo obligan a pedir con puntuaciones altas para evitar que el crupier le gane, al no pasarse, si se plantaba con menos de 17.

Dicho de forma equivalente, cada vez que sale una carta baja crece la ventaja del jugador porque aumenta el porcentaje de cartas altas que quedan en el sabot, y viceversa. Por lo tanto, lo propio es asignarle un valor positivo a las cartas bajas (porque cuando salen dejan el sabot en mejores condiciones para los intereses del jugador) y un valor negativo a las cartas altas (por la razón opuesta).

Thorp recogió en ‘Beat the Dealer’ que el (entonces) mejor sistema de juego le daba un 0,13 % de ventaja al jugador con un mazo completo; es decir, un mazo de 52 naipes con sus cuatro palos completos. Si en ese mazo se suprimen los ases, la expectativa del jugador pasa a ser del $-2,42\%$. Un mazo sin seises y con todas las otras cartas sitúa la ventaja del jugador en un $2,4\%$.

Thorp no indica en ‘Beat the Dealer’ de qué forma determinó estos porcentajes, pues su obra no es un artículo destinado a una publicación científica sino un texto para divulgar su sistema ganador. Cabe pensar que lo hizo mediante cálculos estadísticos tras simular numerosas series de manos en las que quitaba o añadía cartas de determinados valores. Como veremos en su momento, calcular

directamente las probabilidades requiere un buen ordenador, sobre todo si se quiere realizar en tiempo real (de otra forma carece de utilidad para jugar en un casino e incluso para realizar simulaciones en un tiempo razonable).

En cualquier caso, el efecto de extraer una determinada carta del sabot sobre la ventaja del jugador depende de tres variables:

- Composición del sabot en ese momento.
- Puntos del jugador.
- Carta del crupier.

Peter Griffin² cita, a título de ejemplo en su libro ‘The Theory of Black-jack’³, el efecto de quitar cada uno de los diez posibles valores de las cartas⁴ en un mazo completo cuando el jugador tiene un 16 y decide pedir frente a un diez del crupier.

A	2	3	4	5	6	7	8	9	T
-0,5	-0,3	-0,8	-1,7	-2,6	1,7	-0,7	0	0,6	1,2

Tabla 2.5. Efecto, en porcentaje, de extraer cada uno de los valores de las cartas con un 16 frente a un diez, según Peter Griffin.

De acuerdo con la tabla 2.5, si un jugador tiene un 16 y el crupier un 10, sus expectativas de ganar empeoran un 0,5 % si la baraja no tiene Ases y mejoran un 1,2 % si carece de dieces. Los dieces provocan que el jugador se pase si pide con 16 y le cae uno, mientras que los Ases lo ayudan a sumar sin pasarse. Se puede hacer un razonamiento análogo con las otras cartas.

¿Qué debe hacer un jugador a la vista de esta tabla en el sentido de seguir la estrategia básica o tomar otra decisión? Según la tabla 2.2, lo correcto con un 16 frente a un diez es plantarse. Sin embargo, tal decisión parte de que jugamos con un mazo completo, pues la estrategia básica no tiene en cuenta la composición del sabot, distinta cada vez que se extrae una carta.

“La estrategia básica es lo mejor para jugar contra un mazo completo”, afirma Throp. “Pero si faltan algunas cartas (porque ya han salido) y sabemos algo acerca de ellas, generalmente podemos mejorar la estrategia básica. Como simple, aunque no realista, ejemplo supongamos que en sabot sólo quedan cartas de valor cuatro o menor a cuatro. Entonces el jugador debería pedir con un 17 duro (en contraposición con el 17 blando o 7-17, al haber un as con valor 11)

² Peter A. Griffin fue profesor de Matemáticas en un instituto de enseñanza media de California.

³ Uno de los mejores estudios matemáticos sobre el Blackjack.

⁴ Recordamos que hay 13 cartas en cada palo de la baraja francesa pero sólo 10 valores, ya que el Valet, la Reina y el Rey valen 10.

independientemente de la carta del crupier, pese a que la estrategia básica le recomienda plantarse incluso con un 12 si la banca tiene 4, 5 o 6".

Utilizando una idea similar, previamente el propio Thorp había establecido unos índices para modificar la estrategia básica en función de las cartas que aún estaban en el sabot. Mediante simulaciones analizadas posteriormente con métodos estadísticos, este matemático estableció la tabla 2.6 en la cual se recoge el efecto de extraer cada carta de un determinado valor de un mazo sobre la ventaja para el jugador del mejor sistema. Los valores están dados en porcentajes.

Q(1)=0	Q(2)=0	Q(3)=0	Q(4)=0	Q(5)=0	Q(6)=0	Q(7)=0	Q(8)=0	Q(9)=0	Q(10)=0
-2,42	1,75	2,14	2,64	3,58	2,40	2,05	0,43	-0,41	1,62

Tabla 2.6. Porcentaje a favor, o en contra, del jugador si se elimina cada una de las cartas de la baraja

$Q(1) = 0$ significa que no hay ases en el sabot, $Q(2) = 0$ indica que no hay doses, etcétera.

Thorp estableció que si no hay Ases pero tampoco cuatros, la ventaja del mejor sistema sería

$$-2,42\% + 2,64\% = 0,22\%$$

a favor del jugador. Además, por una aproximación lineal, si privar al mazo de los cuatro sietes supone un aumento del 2,05 % para el sistema que se esté utilizando (ya sea uno sustentado en la estrategia básica o cualquier otro), por aproximación lineal extraer un solo siete supone una ventaja añadida de $2,05\%/4 \approx 0,51\%$. Sumando los valores individuales de los naipes que han salido se puede saber, obviamente de forma aproximada, cuál es la ventaja del jugador en cada momento. Esto es de suma importancia para el cuarto nivel de juego (la modificación de la cantidad de dinero apostado antes de comenzar cada mano), que veremos en la siguiente sección. Previamente añadimos la parte final, y esencial, de la modificación de la estrategia básica con el recuento de cartas.

Incluso con una calculadora de bolsillo cuesta un poco sumar los índices decimales de las cartas que han salido en tiempo real; es decir, a la velocidad normal en que se desarrolla el juego en una mesa de Blackjack. Además, lo reiteramos, los dispositivos para contar cartas están rigurosamente prohibidos en los casinos. Como el objetivo de Thorp era encontrar un sistema ganador para el jugador que se pudiera utilizarse en las mesas de Blackjack, su primer paso fue redondear estos índices decimales al número entero más próximo. Así, el 1,7 de retirar los cuatro treses se convertía en un 2 %.

Esta simplificación no le pareció a Thorp suficiente. En realidad, ajustó los índices decimales sólo a tres valores: $-1, 0, +1$. Teniendo siempre como referencia la tabla 2.6, creó el sistema de recuento *Hi-Lo* (altas y bajas) mediante

el cual modificaba tanto algunas decisiones (las más críticas o de frontera) de la estrategia básica, como la cantidad apostada en cada mano.

El *Hi-Lo* es un sistema de un nivel porque el valor de los índices va de -1 a $+1$. Posteriormente surgieron sistemas de dos y hasta tres niveles, como el *Uston avanzado*, descrito por Ken Uston en ‘Million Dollar Blackjack’. A mayor número de niveles, más dificultad en el cálculo y mayor abundancia de errores. Por eso, salvo que el jugador fuese un genio del cálculo mental, los sistemas de recuento más sencillo solían ser tan eficaces en la práctica como los sofisticados.

Como ejemplo, veamos una tabla de índices del *Uston avanzado*:

	2	3	4	5	6	7	8	9	T	A
12	+3	+2	0	-2	-1	H	H	H	H	H
13	-1	-2	-4	-5	-5	H	H	H	H	H
14	-4	-5	-7	-8	-8	H	H	H	H	H
15	-7	-8	-10	-13	-12	H	H	H	H	H
16	-11	-12	-15	-16	-14	H	H	H	H	H
17	S	S	S	S	S	S	S	S	S	S

Tabla 2.7. Tabla de estrategia básica para un mazo modificada por los índices del recuento *Uston avanzado*.

Para utilizar una tabla como la 2.7 es imprescindible tener memorizada previamente la correspondiente tabla de estrategia básica (EB). Por ejemplo, la decisión con un 12 frente a un 3 del crupier es pedir. Sin embargo, si el recuento de carta da un 3 o más, el jugador debe plantarse en vez de pedir. A su vez, la decisión de EB con un 14 frente a un 5 es plantarse, pero si el recuento está en -8 o menos el jugador debe pedir.

Los mejores valores por redondeo para el recuento de un nivel, tipo *Hi-Lo*, son los que aparecen en la tabla 2.8.

A	2	3	4	5	6	7	8	9	T
-1	1	1	1	1	1	0	0	0	-1

Tabla 2.8. Un sistema de recuento de un nivel.

Como la abundancia de dieces y Ases favorecen las expectativas del jugador y su ausencia las disminuyen, si sale un As o un diez se le asigna al sabot un valor -1 . A su vez, dado que la abundancia de cartas bajas perjudica al jugador y favorece al crupier, si sale un cinco se le asigna al sabot un $+1$. Una sucesión de cartas $\{A, 5, T, 2\}$ sumaría $-1 + 1 - 1 + 1 = 0$. El sistema dado en la tabla 2.8 está balanceado porque la suma de los índices asignados a cada uno de los

52 naipes de la barajan es cero (hay 16 dieces y 4 cartas de cada valor distinto a diez).

Un sistema tan simple como el *Hi-Lo* le permitió a Thorp y a un colega suyo ganar 50.000 dólares durante una estancia de pocos días en los casinos de Puerto Rico. Tras esa afortunada experiencia, los casinos cambiaron las normas para anular la ventaja del recuento de cartas.

2.1.4. Estrategia básica y cantidad apostada modificadas por el recuento de cartas

El cuarto nivel en el Blackjack (el más alto que puede asumir una persona normal cuando está en un casino) supone modificar tanto las decisiones de la estrategia básica (EB) como la cantidad de dinero apostada en cada mano en función del recuento de cartas.

Cada carta que sale del sabot produce dos efectos en la forma óptima de jugar:

- a) Modifica las decisiones de la estrategia básica.
- b) Determina la apuesta en la siguiente mano, dependiendo de que el sabot resultante sea más o menos favorable para el jugador.

El segundo efecto es mucho más importante que el primero. Debido a ello, los índices se ajustan con la vista puesta en la modificación de las apuestas, mientras que la alteración de la estrategia básica es secundaria. Sólo modificando las apuestas consigue ventaja un jugador en el Blackjack de seis mazos con las normas habituales en Europa. Justificaremos estas afirmaciones en el próximo capítulo.

Sin entrar en detalles, pues este TFG no pretende ser un tratado sobre cómo jugar al Blackjack, la modificación de la apuesta consiste en jugar lo mínimo permitido (hay un tope mínimo y máximo de apuestas en cada mesa; en general, la amplitud suele ser de 1 a 50 unidades) cuando las expectativas son negativas. Análogamente, si las expectativas son positivas se aumenta la apuesta.

Tras simular cientos de miles de manos en diferentes condiciones, Arnold Snyder⁵ publicó en 1990 unas extensas tablas, tanto para uno como para dos, cuatro y seis mazos, con las ganancias esperadas para el jugador según la amplitud de la variación de la apuesta.

Al margen de su aspecto matemático, la variación de la apuesta en función de los naipes que han salido es un asunto delicado pues es lo primero en lo que se fijan los inspectores de los casinos para detectar si un cliente está contando cartas.

⁵ Autor de ‘Blackbelt in Blackjack’, texto en el que propuso un potente sistema de recuento de dos niveles.

El cálculo directo

3.1. Un programa que sustituya a todos los sistemas

Nuestro propósito al plantearnos este TFG fue diseñar un programa informático capaz de jugar contra el crupier tomando las decisiones adecuadas carta por carta. Para ello utilizaría tres datos:

- a) La composición del sabot (al conocer las cartas que han salido, se sabe cuáles son las que quedan).
- b) La carta del crupier.
- c) La puntuación del jugador tanto con las dos cartas iniciales como con las sucesivas que va pidiendo hasta plantarse o pasarse.

Un programa de este tipo no es utilizable en un casino al prohibirlo las normas. Sin embargo, puede servir como una herramienta de laboratorio para dilucidar cuál es la máxima ventaja que puede conseguirse en el Blackjack, considerando que cualquier jugador humano, utilizando cualquiera de los sistemas de recuento, nunca alcanzará la eficacia de una máquina.

Nada más comenzar a escribir las primeras funciones de este programa nos encontramos con una tarea ardua. El Blackjack parece un juego sencillo, y en esencia lo es, pero sus reglas conducen a situaciones complejas a la hora de programar el desarrollo de una partida. Pronto supimos por qué los matemáticos que abordaron el problema de encontrar sistemas ganadores, en los ya citados años sesenta del siglo pasado, optaron por las simulaciones y la estadística en vez del cálculo de probabilidades.

Algunos casinos de Nevada (principalmente en Las Vegas y Reno) ofrecen algunas opciones adicionales en el Blackjack que aumentan ligeramente la ventaja del jugador. La banca compensa la pérdida de ganancias con un mayor número de clientes atraídos por una normas menos lesivas para sus bolsillos. Por ejemplo, existe la opción de ‘rendirse’: si tras recibir las dos primeras cartas y ver la del crupier el jugador considera que su mano es claramente perdedora, renuncia a

seguir jugando y pierde sólo la mitad de la apuesta. Otra opción es premiar con una paga adicional el conseguir 21 con tres siete, o pedir seis o más cartas sin pasarse.

En el estudio que realizamos a partir de este momento prescindimos de estas opciones suplementarias porque no suelen ofrecerlas sino los casinos de las dos citadas ciudades norteamericanas, y no en todas las mesas (se suele restringir bastante el tope de las apuestas en dichas mesas para no ponerle la partida fácil a los jugadores profesionales).

En definitiva, nos limitaremos a un Blackjack de seis mazos (cuando utilizemos un número distinto de mazos lo especificaremos), con las normas usuales en Europa (de nuevo especificaremos, llegado el caso, si añadimos alguna regla adicional). Esto conlleva a que, según sea la puntuación del jugador, la carta del crupier y las cartas que quedan en el sabot, las funciones programadas puedan tomar las siguientes decisiones de forma que la esperanza matemática en cada mano sea máxima:

- a) Pedir carta o plantarse.
- b) Separar las dos primeras cartas si son del mismo valor.
- c) Doblar, o no, si el jugador suma 9, 10 u 11 con las dos primeras cartas.
- d) Asegurar si el crupier tiene un as.
- e) Antes de comenzar cada mano, detectar si el sabot es favorable para el jugador para que éste incremente la apuesta.

Veremos cada uno de estos apartados en la siguiente sección salvo el quinto, cuya complejidad mediante el cálculo de probabilidades exacto excede el ámbito de este TFG.

3.2. Un problema de detención óptima... y algo más

3.2.1. ¿Dónde parar?

Como hemos señalado anteriormente, la decisión del jugador de Blackjack de plantarse o seguir pidiendo cartas es un problema clásico de detención óptima, con la circunstancia añadida, a su favor, de que el cálculo de probabilidades le da una respuesta inexorable sobre lo que debe hacer. No está en un campo de batalla, en el que un general debe decidir si sus tropas siguen avanzando, se detienen o retroceden, o en el parqué bursátil enfrentado al dilema de precisar hasta cuándo debe mantener un conjunto de acciones sin venderlas, ya sea porque están subiendo y quiere enajenarlas antes de que descienda su precio, o porque están bajando y desea cortar las pérdidas. En ambos ejemplos, así como en muchos más, se pueden construir modelos matemáticos y luego aplicar o bien el cálculo de probabilidades, o bien series de simulaciones para obtener conclusiones sobre lo que se debe hacer en el futuro.

En el caso del Blackjack es posible calcular cuándo es mejor, desde el punto de vista de maximizar la esperanza matemática, pedir o plantarse. La mala noticia es que este cálculo resulta demasiado laborioso. Lo es para abordarlo no sólo con lápiz y papel, sino incluso con un ordenador de sobremesa o portátil por muy potente que sea.

El coste computacional del problema se dispara en la decisión de separar dos cartas del mismo valor.

La decisión de doblar con 9, 10 u 11 es bastante más manejable, siempre en términos de tiempo necesario para el cálculo, si se dispone de una función eficaz para pedir o plantarse.

Asegurar la apuesta se reduce a un simple cálculo de probabilidades.

Aumentar la apuesta antes de comenzar cada mano si el sabot es favorable es una decisión muy difícil de abordar mediante el cálculo de probabilidades.

Veamos un ejemplo antes de pasar a la siguiente sección.

Imaginemos que en un sabot de seis mazos el jugador tiene 16 puntos frente a un diez del crupier. Si pide, hay cinco valores que lo pasarían: el 6, 7, 8, 9 y T. Es decir, hay 192 cartas de 312 (suponiendo que las seis barajas estén completas) que lo pasarían. A su vez, cinco valores, el As, 2, 3, 4, 5, mejorarían su situación sin que llegase a los fatídicos 22 o más puntos.

Como el sabot va cambiando a medida que avanza la partida y el jugador (en este caso un ordenador) conoce cuáles son las cartas que quedan porque lleva la cuenta exacta de las que han salido, bastaría con realizar un mero cálculo de probabilidades:

- a) Si $P(X = \{6, 7, 8, 9, T\}) > P(X = \{A, 2, 3, 4, 5\})$ debe plantarse.
- b) Si $P(X = \{A, 2, 3, 4, 5\}) > P(X = \{6, 7, 8, 9, T\})$ debe pedir.
- c) Si $P(X = \{6, 7, 8, 9, T\}) = P(X = \{A, 2, 3, 4, 5\})$ le resulta indiferente pedir o plantarse.

Sin embargo, esto es sólo una primera aproximación. El jugador pierde automáticamente si al pedir se pasa, cierto, pero no pasarse no es una condición suficiente para ganar (es necesaria pero no suficiente). Puede obtener un 18 y ser derrotado por un 20 del crupier. Además, no está obligado a plantarse con 18, si calcula que le resulta más ventajoso arriesgarse a pedir al menos una carta más. El cálculo de probabilidades no es tan simple como parecía a priori.

Sigamos con este ejemplo. Supongamos que el jugador ha pedido con 16 y ha obtenido un 3, con lo cual tiene ahora 19 puntos. ¿Debe arriesgarse a pedir otra vez o le conviene plantarse?

- Si se planta gana si el crupier obtiene 17, 18 o se pasa, y pierde si el crupier llega a 20, 21 o logra un blackjack.
- Si pide puede llegar a 20, a 21 o pasarse.
 - Si llega a 20, vuelve a calcular la probabilidad de ganar o perder plantándose o pidiendo.

- Si llega a 21 no puede pedir más cartas (cualquier carta lo pasaría). Sólo perdería si el crupier consigue un blackjack (Bj). Ganaría si la banca logra de 17 a 20 puntos y empataría (mano nula) si llega a 21.

Por lo tanto, debe conocer en todo momento cuáles son las probabilidades del crupier de conseguir 17, 18, 19, 20, 21, BJ o 22, pues la banca, obligada a pedir con menos de 17 puntos, sólo puede tener una de estas puntuaciones. (En adelante denotaremos con ‘22’ cualquier puntuación, tanto del crupier como del jugador, superior a 21. Un ‘22’ significa que el jugador o el crupier se ha pasado. BJ indica que el jugador o el crupier, según de quien se trate, ha conseguido un blackjack(Bj): un 21 únicamente con dos cartas).

3.3. Las probabilidades del crupier

Supongamos que el crupier tiene 16 puntos. Como no ha llegado a 17, debe pedir. Sea sb un vector cuyas componentes indican el número de cartas de cada valor que hay en el sabot. Es decir,

$$sb = (sb(1), sb(2), \dots, sb(9), sb(10)) \quad (3.1)$$

siendo $sb(1)$ el número de ases, $sb(2)$ el número de doses, etc. Las probabilidades de que el crupier obtenga 17, 18, $\dots$, 21, 22 son

$$P(X = \{17\}) = \frac{sb(1)}{\sum sb}, \dots, P(X = \{21\}) = \frac{sb(5)}{\sum sb},$$

$$P(X = \{22\}) = \frac{\sum_{i=6}^{10} sb(i)}{\sum sb} \quad (3.2)$$

siendo $\sum sb$ la suma de todas las componentes del vector sb y $\sum_{i=6}^{10} sb(i)$ la suma de las componentes comprendidas entre la sexta y la décima, ambas incluidas.

Veamos ahora el caso en el que el crupier tiene 15 puntos. Sus probabilidades de obtener de 17 a 22 con una sola carta adicional son

$$P(X = \{17\}) = \frac{sb(2)}{\sum sb}, \dots, P(X = \{21\}) = \frac{sb(6)}{\sum sb},$$

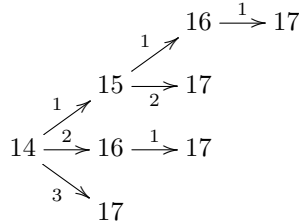
$$P(X = \{22\}) = \frac{\sum_{i=7}^{10} sb(i)}{\sum sb}$$

No obstante, al pedir con 15 puede caerle un As, con lo cual se pondría en 16, puntos con los que debería volver a pedir. Por eso, si el crupier parte de 15 puntos su probabilidad de llegar a 17 en dos pasos es

$$P(X = \{16|15\}) \cdot P(X = \{17|16\}) = \frac{sb(1)}{\sum sb} \cdot \frac{sb(1) - 1}{\sum sb - 1}$$

De forma análoga calcularíamos las probabilidades de 18, 19, ..., 21, 22 a partir de 15.

Si el crupier parte de 14 puntos puede llegar a 17 de cuatro formas distintas:



El árbol anterior se complica bastante si el crupier parte de una puntuación más baja. En una partida real, el jugador juega su mano a la vista de la primera carta del crupier. Por lo tanto, la puntuación inicial de la banca varía entre 2 y 11 (si tiene un as, éste vale 11 porque la suma de naipes no supera los 21 puntos). El árbol con su raíz en 2 ocuparía varias páginas si tratásemos de dibujar un esquema como el anterior.

Se puede programar una función mediante iteraciones para calcular las probabilidades del crupier, teniendo en cuenta que la composición del sabot va cambiando con cada carta extraída. No es igual de probable llegar a 17 desde 16 cuando se ha partido de 14, que llegar a 17 desde 16 cuando, saliendo igualmente de 14, previamente se ha estado en 15. Sin embargo, abordar este problema mediante iteraciones resulta difícil o tedioso. Es mejor recurrir a la recursividad.

Es lo que hemos hecho al programar la función **CrupierProb**, que describimos con detalle en el capítulo 5. De momento nos limitamos a indicar que esta función utiliza como argumentos la puntuación del crupier (c), la posible existencia de un *as* con valor 11 en la mano del crupier ($ca = 1$ indica que hay un *as* con valor 11; $ca = 0$ señala que todos los ases valen 1) y la composición del sabot. Recordemos que sb es un vector cuyas componentes vienen dadas por (3.1).

CrupierProb devuelve, mediante un cálculo recursivo, un vector de probabilidades de las posibles puntuaciones finales del crupier: 17, 18, ..., 21 o Bj . Esta función la utilizaremos como auxiliar en otra, que abordamos en la siguiente sección, para determinar las probabilidades del jugador con cada puntuación, carta del crupier y composición del sabot.

3.3.1. Altas y bajas

Desde el comienzo de este TFG hemos persistido en la idea (hasta este momento una conjetura) de que las cartas bajas favorecen al crupier y, en consecuencia, perjudican al jugador. Aunque no programamos la función **CrupierProb** para dilucidar esta cuestión, podemos utilizarla para ver como actúan las *altas* y las *bajas* en las expectativas de la banca.

Un sabot con todas las cartas:

Si tenemos un mazo completo

$$sb = (4, 4, 4, 4, 4, 4, 4, 4, 4, 16),$$

$$sb(1) = sb(2) = sb(3) = \dots = sb(9) = 4, \quad sb(10) = 16 \quad (3.3)$$

la probabilidad de que el crupier obtenga un *as*, un 2, un 3, ..., un 9 o un *T* (diez) con su primera carta es

$$P(c = A) = \dots = P(c = 9) = \frac{4}{52} \approx 0,0769; \quad P(c = T) = \frac{16}{52} = 0,3076 \quad (3.4)$$

Obviamente, las probabilidades de que se pase con cada una de estas puntuaciones iniciales no son iguales. Utilizando la función `CrupierProb(c,ca,nc,sb)`, considerando un sabot al que le falta uno de los ases tenemos

$$P(c = 22|A) \approx 0,1165; \quad P(c = \{17, \dots, 21, Bj\}|A) \approx 0,8835 \quad (3.5)$$

Estas cantidades hay que multiplicarlas por la probabilidad de que el crupier obtenga un As como primera carta. Hecho esto, tenemos la probabilidad de que la banca puntúe entre 17 y 21, obtenga un *Bj* o se pase:

$$Pp(c = 22|As) \approx 0,0090; \quad Pp(c = \{17, \dots, 21, Bj\}|As) \approx 0,0680 \quad (3.6)$$

De forma análoga podemos calcular

$$Pp(c = 22|x), \quad Pp(c = \{17, \dots, 21, Bj\}|x); \quad x = \{2, 3, \dots, T\} \quad (3.7)$$

y luego calcular la probabilidad ponderada por la probabilidad de que la primera carta del crupier sea *x*. *Bj* denota un blackjack; es decir, un 21 conseguido sólo con dos cartas. *Pp* denota la probabilidad ponderada por la probabilidad de que haya salido la carta *x*.

Con las operaciones necesarias, cuyos detalles omitimos porque son una mera repetición de lo anterior, determinamos las probabilidades a priori de que el crupier se pase si debe jugar con un mazo completo, considerando que el jugador no participa en la partida (es decir, no distorsiona el sabot al pedir sus propias cartas). Así llegamos a que las probabilidades de que el crupier se pase, o no, al jugar con un mazo completo, considerando todas sus posibles primeras cartas, son

$$P(c = 22|x) \approx 0,2836; \quad P(c = \{17, \dots, 21, Bj\}|x) \approx 1 - 0,2866 = 0,7164 \quad (3.8)$$

Un sabot pobre en cartas altas:

Si realizamos las mismas cuentas para un sabot (un mazo, pues seguimos considerando sabots formados por un solo mazo) con la mitad de nueves y la mitad de dieces que uno normal ($sb(9) = 2$, $sb(10) = 8$), los resultados mostrados en (3.8) cambian a

$$P(c = 22|x) \approx 0,2451; \quad P(c = \{17, \dots, 21, Bj\}|x) \approx 0,7549 \quad (3.9)$$

Al comparar (3.9) con (3.8) vemos que el crupier tiene menor probabilidad de pasarse, y consecuentemente más probabilidad de sumar entre 17 y 21 o conseguir un Bj , cuando quitamos cartas altas del mazo. Dicho de otra forma, cada carta alta que sale del mazo lo empobrece para el jugador. Por eso en los sistemas de recuento se le asigna un valor -1 o -2 a un 9 o un 10 que han salido del sabot.

Un sabot pobre en cartas bajas:

En un sabot pobre en cartas bajas la proporción de cartas altas está por encima de la media de un sabot (mazo) normal o completo. Recalculamos lo anterior para un sabot pobre en cartas bajas. Ahora hacemos

$$sb = (sb(1) = 4, sb(2) = 4, sb(3) = 2, sb(4) = 2; sb(5) = 2, \\ sb(6) = 2, sb(7) = 4, sb(8) = 4, sb(9) = 4, sb(10) = 16) \quad (3.10)$$

y obtenemos

$$P(c = 22|x) \approx 0,2596; \quad P(c = \{17, \dots, 21, Bj\}|x) \approx 0,7404 \quad (3.11)$$

siendo x cualquier carta entre un as y un 10. Las probabilidades dadas en (3.11) indican que una mayor proporción de cartas altas no tienen por qué perjudicar al crupier. Al contrario, como hemos visto con el sabot dado en (3.10), puede tener un menor riesgo de pasarse (consiguientemente, mejores expectativas de sumar más de 17 sin pasar de 21) con un sabot parco en cuatro, cinco o seises.

El siguiente paso es comprobar si un sabot con mayor proporción de cartas altas favorece más al jugador que al crupier. Para ello necesitamos una función más complicada, y con mucho mayor consumo de tiempo de cálculo, que la `CrupierProb`

3.4. La mejor decisión del jugador en cada instante

Programar una función que calcule las probabilidades del crupier de obtener entre 17 y 21 o conseguir un Bj es relativamente sencillo, siempre que se

haga por recursividad, pues se sabe de antemano hasta cuándo pide y dónde está obligado a parar.

El caso del jugador es distinto: puede pedir siempre que lo desee, salvo que haya llegado a un 21 o un *Bj*, y también plantarse cuando le apetezca. Además, como hemos visto anteriormente, podría tener la opción de doblar la apuesta, separar un par de cartas iniciales o asegurar.

Vayamos por partes.

3.4.1. Pedir o plantarse

Empecemos con un ejemplo. Supongamos que el jugador tiene 19 puntos frente a un 8 del crupier. Si se planta con 19,

- Gana si el crupier obtiene 17, 18 o se pasa (es decir, si la banca suma 17, 18 o 22, con la notación que hemos adoptado anteriormente).
- Pierde si el crupier llega a 20 o 21.
- Empata (mano nula) si el crupier consigue 19.

Si decide pedir, sus expectativas son:

- Pierde si se pasa (recibe una carta con valor 3 o superior).
- Pide y recibe un $A=1$, con lo cual puede volver a pedir o plantarse.
 - Si pide, puede recibir un $A=1$ (con lo que llega a 21) y esperar a que juegue el crupier. Si se pasa, pierde inmediatamente.
 - Si se planta, queda a expensas de lo que consiga el crupier.
- Pide y recibe un 2, con lo cual llega a 21 y espera a lo que haga el crupier. Sólo podría perder si el crupier consigue un *Bj*, lo cual es imposible en este caso porque la primera carta de la banca no es un diez ni un *as*. Por lo tanto, con 21 frente a un 8 del crupier lo peor que podría ocurrirle es que empate la mano.

El siguiente árbol con raíz en 19 aclara lo que acabamos de exponer.

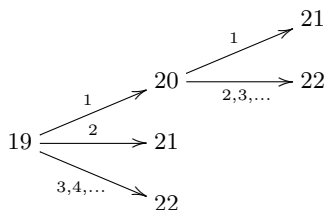


Figura 3.1. Árbol con las posibilidades del jugador cuando tiene 19 puntos.

Para decidir qué hace en los nodos en los que todavía puede pedir (está obligado a plantarse cuando tiene 21 o Bj), el jugador necesita conocer las probabilidades del crupier en cada momento. Para ello puede utilizar la función **CrupierProb**, la cual le devuelve un vector m de 22 componentes.

- $m(1)$ indica la probabilidad de que el crupier logre un Bj .
- Desde la segunda a la decimosexta (2 a 16) las componentes son nulas porque el crupier no se detiene hasta que llega a 17 puntos:

$$m(2) = m(3) = \dots = m(16) = 0$$

- De la decimoséptima a la vigésimo primera (17 a 21), las componentes de m indican, respectivamente, la probabilidad de que el crupier sume cualquiera de esos puntos.
- La componente vigésimo segunda ($m(22)$) recoge la probabilidad de que el crupier se pase.

Obviamente, el vector m hay que recalcularlo en el nodo 20 del árbol de la figura 3.1, pues el sabot ha cambiado con la carta que ha recibido el jugador. De nuevo, avancemos paso a paso:

- Si el jugador se planta con 19, sus probabilidades de ganar, que denotaremos por $gS(19)$, y las de perder, que designamos $pS(19)$, son

$$gS(19) = m(17) + m(18) + m(22); \quad pS(19) = m(20) + m(21) + m(1)$$

En este caso $m(1) = 0$ porque, como hemos señalado, el crupier no puede tener un Bj a partir de un 8 inicial.

- Si decide pedir caben varias posibilidades:
 - Con 19 recibe un as y llega a 20.
 - Se planta con 20. Su probabilidad de ganar es $gS(20) = m'(17) + m'(18) + m'(19) + m'(22)$. La de perder es $pS(20) = m'(21) + \underbrace{m'(1)}_0$.
- Utilizamos m' en lugar de m porque el vector de probabilidad del crupier ha cambiado.
- Pide con 20. Su única posibilidad de ganar es obtener un $A = 1$ y llegar a 21. En este caso, $gH(20) = m''(17) + \dots + m''(20) + m''(22)$. Pierde, en cambio, si recibe una carta de valor 2 o superior. Por lo tanto, su probabilidad de perder es

$$pH(20) = \frac{\sum_2^{10} sb(i)}{\sum sb}$$

con $\sum_2^{10} sb(i)$ la suma de todas las cartas del sabot igual o mayor que 2 y $\sum sb$ la suma de todas las cartas del sabot.

- Con 19 recibe un 2 y llega a 21. Ya no puede pedir. Gana si el crupier suma 20 o menos. Nunca pierde. En el peor de los casos, empata.

Para tomar la decisión óptima con 19 puntos, el jugador debe comparar su esperanza matemática de plantarse

$$E(S) = gS(19) - pS(19)$$

con la de pedir

$$E(H) = gH - pH.$$

Si $E(S) > E(H)$, debe plantarse. Si $E(S) < E(H)$, debe pedir. Si $E(S) = E(H)$, puede pedir o plantarse indistintamente. En lo sucesivo, si $E(S) = E(H)$ elegimos por defecto la opción de plantarnos.

La dificultad de este planteamiento radica en que para saber cuál es la opción óptima en el nodo 19 del árbol 3.1 primero debe saber cuál es la decisión óptima en el nodo 20.

Por añadidura, estamos en un caso trivial. Ningún sistema ganador en el Blackjack, tanto de recuento como de estrategia básica, se plantea pedir con 19. En realidad es raro, salvo un sabot construido a propósito, que pedir sea la mejor opción con un 19 en mano dura (una combinación de cartas en la que no existe un $A = 11$ reconvertible en $A = 1$ si el jugador se pasa). Hemos elegido este ejemplo para mostrar la complejidad de optar por la mejor opción utilizando el cálculo de probabilidades. Complejidad que ilustramos, ítem más, con un ejemplo adicional. Cuando el jugador parte de 18 puntos tenemos

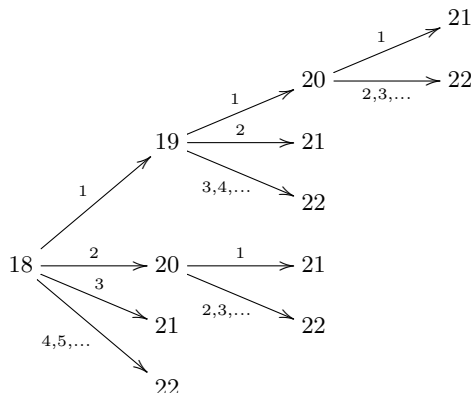


Figura 3.2. Árbol con las posibilidades del jugador cuando tiene 18 puntos.

Nuevamente, la decisión óptima (pedir carta o plantarse) en el nodo 18 depende de lo que ocurre en los nodos hijos 19 y 20 (con 21 y 22 puntos ya no

hay opción a decidir nada); a su vez, la decisión óptima en los nodos 19 y 20 depende de la decisión en el nodo 20, hijo del 19 y nieto del 18.

Si en vez de comenzar desde una puntuación inusualmente alta (por lo que respecta a decidir) como 18, partimos de 12 o 13, e incluso de 6 o 7 en el caso de que el jugador haya separado el primer par de cartas, el árbol de la figura 3.2 crece desmesuradamente.

En este último ejemplo, de 18 hay que ir a 19, de 19 a 20 de 20 a 21, de 21 a 20, de 20 a 22, de 22 a 20, de 20 a 19, de 19 a 21, etc. En cada nodo decisorio, insistimos, hay que recalcular el vector m de probabilidades del crupier y guardar los datos obtenidos para utilizarlos a medida que se avanza y retrocede por las ramas del árbol.

Programar una función que haga esto de forma iterativa es posible, desde luego, pero a costa de mucha paciencia y muchísimas líneas de código. La recursividad, en cambio, reduce el código necesario a menos de un folio. Eso sí, con un importante inconveniente que luego veremos.

La función JugadorDecide()

Para realizar los cálculos en árboles como los mostrados en las figuras 3.1 y 3.2 hemos programado la función recursiva (detallada en el capítulo 4)

`JugadorDecide(j, ja, n, c, ca, sb)`

que usa como argumentos los puntos del jugador en cada momento (j), la posible existencia de un *as* con valor 21 en la mano del jugador (ja), número de cartas utilizadas por el jugador (n), los puntos del crupier (c), la posibilidad de que le primera carta del crupier sea un *as* ($ca = 1$ en este caso) y la composición del sabot en todo momento. Los parámetros c y ca se mantienen constantes mientras juega el jugador; los demás pueden cambiar con cada carta que pide.

Describiremos esta función con detalle en el capítulo 5. De momento nos limitamos a reseñar que devuelve un vector, que denominamos m_j , con ocho componentes

$$m_j = (A, gA, pA, gS, pS, gH, pH, esperanzaA)$$

que detallamos seguidamente:

- A es la mejor decisión en cada nodo del árbol cuya raíz es la puntuación del jugador.

$$A = \begin{cases} 0 & \text{si la mejor decisión es plantarse.} \\ 1 & \text{si la mejor decisión es pedir.} \end{cases}$$

- gA y pA son las probabilidades de ganar o perder con la mejor decisión.
- gS, pS, gH, pH son las probabilidades de ganar o perder plantándose o pidiendo, independientemente de cual sea la mejor decisión.

- *esperanzaA* es la esperanza matemática de ganar con la mejor decisión.

Con esta función podemos calcular las tablas de estrategia básica (ES), así como los índices que las modifican en función de la composición del sabot. No obstante, el uso de la EB y de los índices es innecesario cuando empleamos `JugadorDecide()` porque la función conoce en cada momento cuál es la composición del sabot y calcula en tiempo real (el tiempo que tiene un jugador para decidir en el casino) las mejores opciones para $j \geq 12$, y casi en tiempo real aquellas para $j < 12$. En el caso de las manos blandas (en las que interviene un *as* con valor 11) cuesta un poco más, en términos de tiempo, calcular la decisión, aunque se puede considerar que lo sigue haciendo en tiempo real para $j \geq 12$; es decir, para la inmensa mayoría de las situaciones críticas que surgen durante una partida usual en una mesa de Blackjack. Lamentablemente, no se puede utilizar en un casino porque en ninguno nos dejarían entrar con un ordenador bajo el brazo.

Ejemplo de una decisión con `JugadorDecide()`

Supongamos que tenemos dieciséis frente a un diez del crupier, lo cual denotamos por 16|10. Vimos en la tabla 2.2, referente a la estrategia básica, que la mejor decisión para el jugador cuando tiene 16|10 es plantarse. La EB no tiene en cuenta la composición del sabot. Sin embargo, los sistemas de recuento de cartas modifican la decisión de 16|10 en función de las cartas que hayan salido.

Casi todos los sistemas de recuento indican que se debe pedir si la cuenta de cartas está en negativo; es decir, si han salido más cartas altas que bajas. En cambio, si la cuenta está en positivo (han salido más cartas bajas que altas), los índices del sistema *Uston avanzado*, recogidos en la tabla 2.7, nos indican que debemos pedir si la cuenta está en -14 o menos y plantarnos si está por encima de esta cifra. El *Uston avanzado* asigna un -3 a cada diez que sale del sabot. Por lo tanto, para que nos resulte más ventajoso pedir que plantarnos con un 16|10 tendrían que haber salido los 4 dieces del mazo y al menos 2 nueves (este sistema asigna un -1 al nueve).

Arnold Snyder implementó en ‘Blackbelt in Blackjack’ un potente sistema de recuento de dos niveles que denominó *Zen count*. En él se asigna un -2 al diez. Según su tabla de índices, calculada por simulaciones de millones de manos con distintas composiciones del sabot, el jugador debe pedir con 16|10 si la cuenta está en negativo. Es decir, debe pedir si en el sabot falta al menos un *as* (el *Zen Count* asigna un -1 al *as*) o un diez.

Si llamamos a la función `JugadorDecide` con un sabot equilibrado (aquel cuya suma de altas y bajas es cero) nos indica que debemos plantarnos. Si sacamos del sabot todos los Ases (la cuenta estaría en -4 según el sistema de Snyder), `JugadorDecide` nos recomienda que sigamos platándonos. En cambio, es suficiente con que extraigamos un diez para que la recomendación pase de plantarnos a pedir. Lo mismo ocurre si extraemos un 9.

Si quitamos del sabot un diez y un cinco, la recomendación se mantiene en plantarnos: la misma que con un sabot equilibrado porque hemos compensado la eliminación de un diez con la de un cinco. De hecho, la cuenta ha aumentado un poco al sacar un diez y un cinco. Si con la cuenta en cero hay que plantarse, mucho más hay que hacerlo si está por encima de cero, pues esto último indica que la proporción de cartas altas es mayor.

El coste de extraer una carta del sabot no es constante. No es lo mismo perder un *as* en una situación de 16|10 que en otra de 9|10. Los sistemas de recuento de cartas, además de redondear los decimales al entero más próximo, promedian estos valores pues ya resulta bastante arduo para el jugador recordar que el *as* vale -1 y el diez, por ejemplo, -2 . Pretender que memorice varios índices para cada carta, en función de su puntuación y la del crupier, está más allá de la capacidad mental de la inmensa mayoría de los habitantes de este planeta. Para **JugadorDecide**, en cambio, este cálculo exacto de probabilidades no supone ningún problema, salvo el tiempo empleado para realizarlo cuando se parte de una puntuación del jugador baja ($j \leq 6$).

3.4.2. Doblar con 9, 10 u 11

Para dilucidar si al jugador le conviene doblar la apuesta cuando tiene 9, 10 u 11 ($j = \{9, 10, 11\}$) no es suficiente con la función **JugadorDecide** debido a una regla restrictiva del Blackjack: si el jugador dobla con cualquiera de esas tres puntuaciones sólo recibe una carta adicional. (Podría darse la situación de que doblase con un 9, recibiese un 2 y se quedase con 11, que es una de las mejores puntuaciones para pedir sin riesgo de pasarse y caer cerca de 21).

La función **JugadorDobla(j, ja, n, c, ca, sb)** calcula si resulta ventajosa esta jugada. Primero llama **JugadorDecide** con el fin de obtener la esperanza del jugador si no dobla (sigue el procedimiento normal con una mano que comienza en 9, 10 u 11 puntos), y luego calcula el pago esperado cuando doblando la apuesta recibe una única carta más.

3.4.3. Separar cartas

Cuando un jugador separa cartas, añade una apuesta igual a la primera y sigue con dos manos distintas. El procedimiento es el siguiente:

1. El jugador recibe una carta, el crupier toma la suya y el jugador recibe su segunda carta. Si ambas cartas del jugador tienen el mismo valor, éste puede optar por separarlas. Para ello pone sobre el tapete una nueva apuesta igual a la primera.
2. El crupier le da una tercera carta sobre la primera de las dos que ha separado, de forma que en esa mano la puntuación del jugador es

$$j = j_1 + j_3; \quad j_1 \equiv \text{puntos de la primera carta, } j_3 \equiv \text{puntos de la tercera carta.}$$

3. El crupier le da una cuarta carta sobre la primera de la segunda mano, con lo cual la puntuación en esa mano es

$$j' = j_2 + j_4; \quad j_2 \equiv \text{puntos de la segunda carta}, \quad j_4 \equiv \text{puntos de la cuarta carta}.$$

Obviamente, $j_1 = j_2$ pues en caso contrario el jugador no habría podido separar

A partir de este momento, el jugador gestiona la primera mano considerando no sólo los puntos que tiene en ella, la carta del crupier y la composición del sabot, sino también lo que podría ocurrirle cuando, una vez que haya decidido plantarse o se haya pasado en la primera mano, le toque gestionar la segunda.

En este caso, si aplicamos la recursividad sucesivamente a ambas manos, una vez recorrido hasta el final un árbol como los mostrados en las figuras 3.1 o 3.2 habría que recorrer un segundo árbol, correspondiente a la segunda mano, también hasta el final, retroceder luego hasta su raíz y luego retroceder hasta el nodo del primero del que hemos partido. El procedimiento hay que repetirlo con cada nodo decisorio (un nodo decisorio es todo aquel distinto a 21 o 22) del primer árbol.

Hemos programado la función `JugDecSep(j, ja, n, ji, jai, sep, c, ca, sb)` que realiza el procedimiento que acabamos de describir. Sin embargo, puede tardar horas en calcular la conveniencia, o no, de separar un par de cartas, salvo que utilicemos un sabot muy sencillo que no suele darse en la práctica.

Para solventar este inconveniente hemos recurrido a un procedimiento aproximado. Primero calculamos la esperanza del jugador partiendo de una puntuación

$$j = j_1 + j_2, \quad j_1 = j_2$$

y luego partiendo de j_1 y multiplicando por dos la esperanza obtenida, ya que al separar se dobla la apuesta. Comparando ambas esperanzas, decidimos si conviene separar o jugar sin hacerlo, de acuerdo con las recomendaciones de la función `JugadorDecide()`.

El procedimiento es aproximado porque, al realizar los cálculos de la separación, utilizamos el mismo sabot tanto para la primera como para la segunda mano, cuando en realidad el sabot ha cambiado al iniciar la segunda pues al jugar la primera se han consumido cartas.

En la práctica, esta simplificación no distorsiona excesivamente los resultados por dos motivos:

- a) Al inicio de la partida, después barajar las cartas, el crupier retira cinco cartas ('las quema', en el argot del Blackjack) y las deposita en la pila de descarte sin que nadie vea su valor. Por ello ni siquiera un ordenador puede saber exactamente el valor de las cartas que hay en el sabot mientras se desarrolla la partida.

- b) El sabot, o el mazo, no se juega hasta el final. El crupier introduce una tarjeta en la ristra de cartas que separa una porción final del sabot. Cuando sale esta tarjeta indicadora, el crupier sigue dando cartas pero sólo hasta que concluye la mano en curso. Terminada esta, mezcla todos los naipes e inicia un nuevo sabot.

El porcentaje de cartas que quedan sin salir suele variar entre el 16 y el 25 por ciento del total, según lo que considere conveniente la dirección de cada casino. En algunos casinos, muy cuidadosos con el recuento de cartas, suele colocar la tarjeta de corte en la mitad del sabot. Al quedar la mitad de las cartas sin salir se reduce notablemente la ventaja de los jugadores avezados.

En cualquier caso, la incertidumbre sobre lo que realmente queda en el sabot es mucho mayor a la ocasionada por despreciar los dos, tres, cuatro y a lo sumo, cinco o seis naipes consumidos al jugar una mano. A veces, lo óptimo es enemigo de lo bueno.

3.4.4. Asegurar

El seguro es una apuesta independiente que el jugador puede hacer si la primera carta del crupier es un *as*. El máximo de esta apuesta es la mitad de la apuesta inicial. Si el jugador ha apostado cuatro euros al comienzo de la mano, puede asegurar por un máximo de dos euros. (Podría asegurar sólo un euro, si así lo desea).

En nombre de ‘seguro’ para esta apuesta viene de que si el jugador tiene un *Bj* y el crupier un *as* (con lo cual es candidato también a tener un *Bj*), el jugador puede cobrar su *Bj* como una apuesta sencilla (1 a 1 en vez de 3 a 2). Esta acción (basta con que el lector realice un simple cálculo) es equivalente a asegurar el *Bj*. En cualquier caso, el jugador no puede cobrar por adelantado (como una apuesta sencilla) su *Bj* si el crupier tiene un diez en lugar de un *As*, aunque con un diez también sea candidato a conseguir un *Bj*.

Si el crupier obtiene un blackjack (*Bj*), esto es, recibe un diez como segunda carta, el jugador gana el seguro con independencia de lo que le ocurra con su mano: puede ganar la apuesta inicial y perder el seguro, perder la apuesta inicial y también el seguro o perder la apuesta inicial y ganar el seguro.

Como el seguro paga 2 a 1, la esperanza matemática del jugador es

$$E = 2 \cdot \frac{sb(10)}{\sum sb} - \frac{\sum_1^9 sb}{\sum sb} \quad (3.12)$$

con $sb(10)$ el número de dieces que hay en el sabot, $\sum sb$ el número total de cartas del sabot y $\sum_1^9 sb$ el número de cartas con valor distinto a diez. La función **JugAsegura**(*sb*) realiza este simple cálculo. Evidentemente, no estamos teniendo en cuenta las cartas que puede utilizar el jugador en su propia mano antes de

que le toque jugar al crupier. Sin embargo, salvo que hagamos una estimación de este consumo de naipes, cuya complicación adicional dudosamente compensaría la precisión ganada, la única forma de calcular el seguro es la descrita.

Funciones programadas

En el desarrollo de este TFG hemos programado varias funciones con la finalidad de que el jugador pueda decidir cuál es su mejor opción en todo momento atendiendo, como hemos reiterado, a su puntuación, la carta del crupier y la composición del sabot.

Estas funciones están programadas en Matlab, versión 2017a. Matlab es una abreviatura de MATrix LABoratory. Ofrece un entorno de desarrollo integrado (IDE) bastante cómodo de usar y muy útil a la hora de depurar el código programado.

Como hemos señalado anteriormente, estas funciones no son útiles para jugar en un casino (por la prohibición de utilizar ordenadores), pero sirven como buenas herramientas para analizar algunas características del Blackjack.

4.1. Las funciones

4.1.1. CrupierProb

La función `CrupierProb(c,ca,nc,sb)`, que podemos ver completa en el anexo final de este TFG, es auxiliar a la que recurre en cada nodo `JugadorDecide`. Esta última le pasa los argumentos constantes c y ca (carta del crupier y posible existencia de un As con valor 11 en la mano del crupier) y el sb (composición del sabot) convenientemente modificado.

La función primeramente genera un vector de ceros, denominado m , con 22 componentes destinadas a alojar las probabilidades de las puntuaciones que puede obtener el crupier. Como hemos señalado anteriormente, la componente $m(1)$ indica la probabilidad de que el crupier obtenga un Bj, las $m(17) \dots m(21)$ sus probabilidades de lograr entre 17 y 21 puntos y $m(22)$ su probabilidad de pasarse.

El crupier parte siempre de una puntuación $c < 17$, pues con la primera carta que se da a sí mismo $2 \leq c \leq A = 11$. A este valor inicial c se le suman i valores, con $i = \{2, 3, \dots, 10, A = 11\}$ si $c \leq 10$ o $i = \{A = 1, 2, \dots, 10\}$ si $c \geq 11$.

Mientras $c + i < 17$, la función se llama a sí misma (recurre a sí misma) con

$$c = c + i, \quad ca = \{0, 1\}, \quad nc = nc + 1, \quad sb = sbn$$

siendo $ca = 0$ si todos los Ases valen 1 y $ca = 1$ si hay un As con valor 11. A su vez, sbn es el sabot al que se le han quitado las cartas que han salido desde que el crupier inició su jugada.

La función devuelve el vector m con la la probabilidad de cada puntuación del crupier.

4.1.2. JugadorDecide()

Recuperamos el árbol de la figura 3.2:

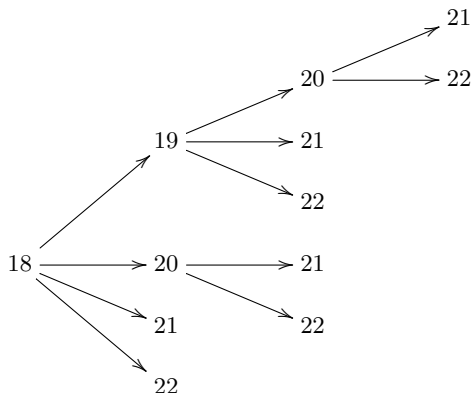


Figura 4.1. Árbol con raíz en 18 puntos.

La función `JugadorDecide(j,ja,n,c,ca,sb)` lo recorre de la siguiente forma indicada en la figura 4.1.

De esta manera, pasa por todos los nodos no más veces de las necesarias para llegar al final de cada rama. Como hemos visto anteriormente, para tomar la mejor decisión en j primero debe conocer la mejor decisión en

$$j + i, \quad i = \{1, 2, \dots, \min(10, 21 - j)\}$$

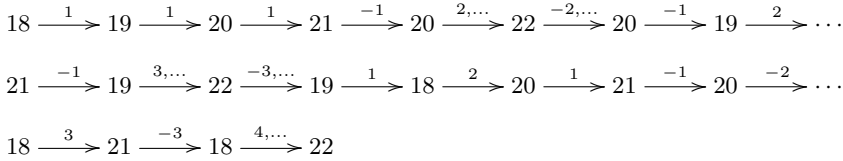


Figura 4.2. Camino recorrido por la función `JugadorDecide()` en el caso del árbol de la figura 4.1. Los índices sobre las flechas indican el valor de la carta consumida para pasar de una puntuación a la siguiente. Los índices negativos indican que una carta de ese valor ha sido devuelta al sabot al retroceder a un nodo anterior. Los índices con puntos suspensivos indican que cualquier carta igual o superior pasan al jugador.

Hemos programado esta función por recursividad, de forma que se llama a sí misma hasta que llega a 21 o 22. Es decir, hasta que el jugador alcanza la máxima puntuación posible o se pasa.

La recursividad tiene la ventaja de simplificar mucho la programación y el inconveniente del mayor gasto computacional. A este consumo de tiempo de la propia función `JugadorDecide` hay que añadir el de `CrupierProb()`, función a la que `JugadorDecide()` llama en cada nodo decisorio (un nodo que no sea 21 o 22).

`JugadorDecide()` devuelve como resultado la mejor opción para el jugador ('0' plantarse, '1' pedir), la probabilidad de ganar y perder con esa elección óptima, la probabilidad de ganar y la de perder tanto si se planta como si pide y su esperanza con la mejor opción.

El código completo de `JugadorDecide()` está en el anexo final.

4.1.3. JugDecSep()

Hemos visto que si las dos primeras cartas que recibe el jugador son del mismo valor puede separarlas y gestionar dos manos independientes.

Si ha decidido separar, lo que haga en la primera mano sólo afecta a la segunda en las cartas consumidas. Sin embargo, antes de separar debe decidir si:

- Le conviene plantarse con la puntuación inicial $j = j_1 + j_2$, $j_1 = j_2$.
- Le conviene pedir con la puntuación inicial.
- Le conviene separar y comenzar a jugar la primera de dos manos distintas con puntuación inicial j_1 . Esta mano la juega con las recomendaciones que le dicta la función `JugadorDecide()`. Una vez concluida esta mano, porque decide plantarse o se ha pasado, sigue con la segunda, cuyo origen es $j_2 (= j_1)$, siguiendo igualmente lo que vaya estableciendo `JugadorDecide`.

Para las decisiones de los apartado (a) y (b) , tiene suficiente con la función **JugadorDecide**. Para el apartado (c) le hace falta una función que, después de recorrer el árbol correspondiente a la primera mano (un árbol similar al de la figura 4.1), siga por la raíz del árbol correspondiente a la segunda mano hasta el final de cada una de sus ramas. Luego debe retroceder hasta la raíz de este segundo árbol y seguir retrocediendo por las ramas del primero. En cada nodo decisorio, tanto del primer como del segundo árbol, la función debe llamar a **CrupierProb**.

Esto lo hace la función **JugDecSep()**, que también recogemos completa al final de este capítulo. Sus argumentos son: *j*, puntuación total del jugador con las dos primeras cartas; *ja*, presencia de un As con valor 11 en la mano inicial del jugador; *n*, número de cartas del jugador; *ji*, valor de la carta inicial del jugador; *jai*, indicador si una de las cartas iniciales es un As. (Si lo es una, forzosamente también lo es la otra); *sep*, indicador de si el jugador está en la primera o en la segunda mano (en la primera, *sep* = 1, en la segunda *sep* = 0); *c*, puntos del crupier; *ca*, indicador de As con valor 11 en la mano del crupier (*ca* = 1 indica que hay un A=11); *sb*, composición del sabot.

Con los recursos informáticos a nuestra disposición sólo hemos podido probar esta función con sabots muy simples. Por ello, en los cálculos hemos recurrido a la simplificación expuesta en el apartado 3.4.3

4.1.4. JugadorDobla()

Es, en esencia, una ampliación y adaptación de **JugadorDecide()** para que contabilice el efecto de que el jugador sólo recibe una carta adicional si decide doblar con 9, 10, 11. Si su esperanza es mayor cuando dobla, lo recomienda. En caso contrario, juega la mano normalmente siguiendo las indicaciones de **JugadorDecide()**. Incluimos su código completo en el anexo final, con los correspondientes comentarios aclaratorios.

4.1.5. JugadorAsegura()

Esta función es tan simple que no requiere comentarios. Su código está en el anexo final.

Conclusiones y posibles ampliaciones

Quisimos en este trabajo indagar hasta qué punto es posible, utilizando estrictamente el cálculo de probabilidades (es decir, sin necesidad de simulaciones y el correspondiente tratamiento estadístico de las mismas), determinar las decisiones óptimas de un jugador de Blackjack atendiendo a su puntuación en cada momento, la carta del crupier y la composición del sabot también en cada instante de la partida

Hemos mostrado que, mediante unas funciones adecuadas, programadas por recursividad, es posible realizar este cálculo. No obstante, los costes en términos de tiempo de computación son enormes, al menos con un ordenador portátil o de sobremesa. Además, como en los casinos están prohibidos los dispositivos de cualquier tipo que ayuden al jugador, las decisiones sustentadas en el cálculo de probabilidades son doblemente inviables.

Las funciones programadas las hemos utilizado para recalcular las tablas de estrategia básica (EB). La EB es un conjunto de normas que debe seguir el jugador de Blackjack para minimizar su desventaja frente a la banca y, en determinadas circunstancias favorables debido a las reglas generosas de algunos casinos norteamericanos, conseguir ventaja. La EB tiene en cuenta la carta del crupier y la puntuación del jugador, pero no (esa es su principal carencia) la composición del sabot. También hemos usado estas funciones para determinar los índices que corrigen, en el sentido de mejorarlas, las recomendaciones de la EB en función de la composición del sabot. Los resultados obtenidos son bastante coincidentes con los determinados, mediante métodos estadísticos, por los matemáticos que han estudiado el Blackjack durante el último medio siglo. En algunos casos, muy pocos, las funciones nos han proporcionado resultados ligeramente distintos. Ello es debido a que las simulaciones dan resultados promediados, mientras que nuestras funciones determinan la decisión óptima contando con todas las cartas que quedan en el sabot.

Recalculamos las tablas de EB y sus índices adjuntos con dichas funciones. Estos resultados no se recogen en la memoria final por falta de espacio.

De cara al futuro

La continuación de este trabajo supone determinar, también mediante el cálculo de probabilidades, si la composición del sabot es favorable para el jugador antes del inicio de cada mano, de forma que éste pueda incrementar su apuesta de ser así. Esto ya se ha hecho con procedimientos estadísticos. Realizarlo con nuestras funciones es una tarea compleja que supera el límite de este TFG.

Igualmente interesante sería comparar los resultados al jugar con nuestras funciones con los obtenidos por métodos tradicionales. Es decir, la EB complementada con el recuento de cartas y la variación de apuesta antes de cada mano.

Las conclusiones que obtendríamos quizá podrían aplicarse a otros campos. No en vano Edward Thorp trasladó, con notable éxito, su sistema ganador en el Blackjack a la toma de decisiones en los mercados bursátiles.

A

Anexo

A.1. Funciones

Las funciones de este TFG las hemos programado en Matlab: una abreviatura de MATrix LABoratory, o ‘laboratorio de matrices’. Como indica su nombre, este lenguaje, junto a su plataforma IDE, es ideal para manipular matrices y resulta muy sencillo programar con él.

A.1.1. CrupierProb()

```
function [ d ] = CrupierProb(c,ca,nc,sb)

% c es la puntuación inicial del crupier; d es el vector de
% probabilidades del crupier.
% ca indica si hay un con valor 11.
% nc: número de cartas tomadas por el crupier.

d = zeros(1,22); % Se genera un vector de 22 componentes,
% todas nulas.

% Las componentes d(17) ... d(21) recogen la probabilidad de
% que el crupier obtenga un 17, un 18, ... o un 21.
% La componente d(22) indica la probabilidad de que el
% crupier se pase.
% La d(1) indica la probabilidad de que el crupier consiga Bj
% .

if c >= 2 && c <= 9 % Caso de que el crupier tenga de 2 a 9
puntos:
    sbn = sb; % El caso del 'as' se trata por separado.
    if sbn(1) > 0 % ¿Hay almenos un 'as' en el sabot?
```

```

    sbn(1) = sbn(1) - 1;
    % Se recurre a la función con 'c' más el As que ha
    % salido, el indicador de 'as' significativo puesto
    % a 1 (si 12 <= c <= 10 un 'as' añadido siempre es
    % significativo), el contador de cartas del
    % crupier incrementado (nc=nc+1) y el sabot
    % actualizado con un As menos.
    d = d + CrupierProb(c+11,1,nc+1,sbn)*sb(1)/sum(sb);
end
% Casos en los que el crupier obtenga entre '2' y '10' al
% pedir.
for i = 2:10
    sbn = sb;
    if sbn(i) > 0 % ¿Hay cartas de cada uno de los
    % valores en el sabot?
        sbn(i) = sbn(i) - 1;
        d = d + CrupierProb(c+i,ca,nc+1,sbn)*sb(i)/sum(sb
        );
    end
end
end
if c == 10 && nc == 1 % c = 10 con una carta.
    sbn = sb;
    if sbn(1) > 0 % ¿Hay Ases en el sabot?
        sbn(1) = sbn(1) - 1;
        d(1) = sb(1)/sum(sb); % El crupier logra Bj.
    end
    for i = 2:10 % El as está contabilizado en el caso
    % anterior.
        sbn = sb;
        if sbn(i) > 0 % ¿Hay cartas de cada uno de los
        % valores en el sabot?
            sbn(i) = sbn(i) - 1;
            d = d + CrupierProb(c+i,ca,nc+1,sbn)*sb(i) / sum(
            sb);
        end
    end
    return
end % Final de c == 10 && nc > 1.
if c == 10 && nc > 1 % c = 10 con más de una carta.
    % La prob. de que salga un As se contabiliza aparte.
    sbn = sb;
    if sbn(1) > 0 % ¿Queda al menos un As en el sabot?
        sbn(1) = sbn(1) - 1;
        d = d + CrupierProb(c+11,ca,nc+1,sbn)*sb(1)/sum(sb);
    end
end

```

```

    for i = 2:10
        sbn = sb;
        if sbn(i) > 0
            sbn(i) = sbn(i) - 1;
            d = d + CrupierProb(c+i,ca,nc+1,sbn)*sb(i) / sum(
                sb);
        end
    end
end % Final de c == 10 && nc > 1.
% c == 11 con una carta.
if c == 11 && nc == 1
    sbn = sb;
    if sbn(10) > 0 % ¿Hay al menos un '10' en el sabot?
        sbn(10) = sbn(10) - 1; d(1) = sb(10) / sum(sb); % El
            crupier logra un Bj.
    end
    for i = 1:9 % El 10 está contabilizado en la condición
        anterior.
        % Como el CPR ya tien e11, todos los 'ases' valen
            '1'.
        sbn = sb;
        if sbn(i) > 0
            sbn(i) = sbn(i) - 1; d = d + CrupierProb(c+i,1,nc
                +1,sbn)*sb(i) / sum(sb);
        end
    end
end % Final c=11 con una carta.
% c == 11 con más de una carta.
if c == 11 && nc > 1
    for i = 1:10
        sbn = sb;
        if sbn(i) > 0
            sbn(i) = sbn(i) - 1; d = d + CrupierProb(c+i,ca,
                nc+1,sbn)*sb(i)/sum(sb);
        end
    end
end % Final de c=11 con más de una carta.
% 12 <= c <= 16
if c > 11 && c < 17
    for i = 1:10
        sbn = sb;
        if sbn(i) > 0 % ¿Hay cartas de cada valor en el sabot
            ?
            sbn(i) = sbn(i) - 1;
            d = d + CrupierProb(c+i,ca,nc+1,sbn)*sb(i) / sum(
                sb);
        end
    end
end

```

```

        end
    end
end % Final 12 <= c <= 16
% De 17 a 21.
if (c >= 17) && (c <= 21)
    d(c) = 1;
    return
end
if c > 21 && ca == 1 % Más de '21' con un As significativo:
    d = d + CrupierProb(c-10,0,nc,sb);
end
if c > 21 && ca == 0
    d(22) = 1;
    return
end
end % Final función.

```

A.1.2. JugadorDecide()

```

function decision = JugadorDecide(j, ja, n, c, ca, sb)
% Argumentos:
% j: Puntos del jugador (siempre menor o igual que 21).
% ja: Indicador de 'as' significativo en la mano del jugador.
% c: Puntos del crupier.
% ca: Indicador de 'as' significativo en la mano del crupier.
% sb: Sabot actual.
% La función devuelve el vector [A,gA,pA,gS,pS,gH,pH,
    esperanzaA].
% A: Decisión óptima: 0='Plantarse', 1='Pedir'
% gA: Probabilidad de ganar si se toma esa decisión.
% pA: Probabilidad de perder si se toma esa decisión.

m = CrupierProb(c, ca, 1, sb); % Vector de probabilidades del
    crupier.
% Vectores de probabilidades de ganar o perder.
gpS = [0,0]; % vector [ganar si S, perder si S]
gpH = [0,0]; % vector [ganar si H, perder si H]
if j == 21 && n == 2 %Si el jugador tiene BJ nunca pierde.
    gpS(1) = sum( m(17:21) ) + m(22); gpS(2) = 0;
    gpH(1) = 0; gpH(2) = 1;
elseif j == 21 % Si el jugador tiene 21 ya no puede pedir.
    gpS(1) = sum(m(17:20)) + m(22); gpS(2) = m(1);
    gpH(1) = 0; gpH(2) = 1;
elseif 10 < j && j < 21
    gpS(1) = sum( m(17:j-1) ) + m(22);
    gpS(2) = sum( m(j+1:21) ) + m(1);

```

```

for i = 1:min(21-j,10) % Recibe una nueva carta i con la
    que no se pasa.
    if sb(i) > 0
        sbn = sb; sbn(i) = sbn(i) - 1;
        res = JugadorDecide(j+i, ja, n + 1, c, ca, sbn);
        gpH = gpH + res(2:3) * sb(i)/sum(sb);
    end
end
k = 22 - j; % Con cartas de valor k (o mayores) se pasa
if (k <= 10) && (ja==0) % La mano es dura; se pasa:
    gpH(2) = gpH(2) + sum(sb(k:10))/sum(sb);
end
if (k <= 10) && (ja==1) % La mano es blanda.
    for i = k:10
        if sb(i) > 0
            sbn = sb; sbn(i) = sbn(i) - 1;
            res = JugadorDecide(j + i - 10, 0, n + 1, c,
                ca, sbn); % Al pasarse el jugador y
                reajustar la puntuación, la manos se
                convierte en dura.
            gpH = gpH + res(2:3) * sb(i)/sum(sb);
        end
    end
end
elseif j <= 10
    gpS(1) = sum( m(17:j-1) ) + m(22);
    gpS(2) = sum( m(j+1:21) ) + m(1);
    if sb(1) > 0 %Si hay al menos un 'as' en el sabot.
        sbn = sb; sbn(1) = sbn(1) - 1;
        res = JugadorDecide(j + 11, 1, n + 1, c, ca, sbn);
        gpH = gpH + res(2:3) * sb(1)/sum(sb);
    end
    for i = 2:10 % Recibe una nueva carta i con la que no se
        pasa.
        if sb(i) > 0
            sbn = sb; sbn(i) = sbn(i) - 1;
            res = JugadorDecide(j + i, ja, n + 1, c, ca, sbn)
            ;
            gpH = gpH + res(2:3) * sb(i)/sum(sb);
        end
    end
    elseif j > 21 && ja == 0
        gpS = [0,1]; gpH = [0,1];
end
esperanzaS = gpS(1) - gpS(2); esperanzaH = gpH(1) - gpH(2);

```

```

if (esperanzaS >= esperanzaH)
    decision = [0, gpS(1), gpS(2), gpS(1), gpS(2), gpH(1),
                gpH(2), esperanzaS];
else
    decision = [1, gpH(1), gpH(2), gpS(1), gpS(2), gpH(1),
                gpH(2), esperanzaH];
end
end % Final de la función.

```

A.1.3. JugDecSep(j, ja, n, ji, jai, sep, c, ca, sb)

```

function decision = JugDecSep(j, ja, n, ji, jai, sep, c, ca,
    sb)
% Argumentos:
% j: Puntos del jugador (siempre menor o igual que 21)
% ja: Indicador de As significativo en la mano del jugador
    (1/0).
% n: Número de naipes que en total suman 'j'.
% ji: Naipe inicial separado (de 2 a 11)
% jai: 1 si el inicial es As, 0 en otro caso.
% sep: 1 si al llegar al final de una mano debe comienza otra
    con 'j'='ji', n=1, sep=0.
% c: Puntos del crupier.
% ca: Indicador de As significativo en la mano del crupier.
% sb: Sabot actual.
% La función devuelve el vector [A, gA, pA, gS, pS, gH, pH,
    gSep, pSep].
% A: Decisión óptima: 0='Plantarse', 1='Separar y pedir'
% gA: esperanza de las ganancias si se toma la decisión A.
% pA: esperanza de las pérdidas si se toma la decisión A.
    Esperanza de ganancia neta: 'gA-pA'.
m = CrupierProb(c, ca, 1, sb); % Vector de probabilidades de
    los puntos del crupier: [Bj,2,...,22]
gpS = [0, 0]; gpH = [0, 0]; % Vectores de esperanzas de
    ganancias y de pérdidas.
gpSep = [0, 0]; % Se crea este vector inicializado a 0.
sumsb = sum(sb); % Número de naipes en el sabot.
if j == 21
    gpH = [0, 1];
    if n == 2 % Si el jugador tiene Bj nunca pierde.
        gpS = [sum( m(17:21) ) + m(22), 0];
    else % Si el jugador tiene 21 ya no puede pedir. Gana
        si el crupier tiene menos de 21 y pierde si el
        crupier consigue un BJ (m(1) > 0)
        gpS = [sum( m(17:20) ) + m(22), m(1)];
    end
end

```

```

if sep == 1 % Ahora comienza otra mano, con j = ji + un
segundo naipe, As o no
    if sb(1) > 0
        sbn = sb; sbn(1) = sbn(1) - 1;
        if jai == 1 % La mano original separada era
            blanda, sumo 1
            res = JugDecSep(ji + 1, 1, 2, ji, jai, 0, c,
                ca, sbn);
        else % La mano original separada era dura, sumo
            11 y pasa a blanda
            res = JugDecSep(ji + 11, 1, 2, ji, jai, 0, c,
                ca, sbn);
        end
        gpS = gpS + res(2:3) * sb(1)/sumsb;
    end
    for m = 2:10
        if sb(m) > 0
            sbn = sb; sbn(m) = sbn(m) - 1;
            res = JugDecSep(ji + m, jai, 2, ji, jai, 0, c,
                ca, sbn);
            gpS = gpS + res(2:3) * sb(m)/sumsb;
        end
    end
end
elseif 10 < j && j < 21
    gpS(1) = sum( m(17:j-1) ) + m(22); gpS(2) = sum( m(j
        +1:21) ) + m(1);
    if sb(1) > 0 % Recibe una nueva carta i con la que no se
        pasa, As=1 o no.
        sbn = sb; sbn(1) = sbn(1) - 1;
        res = JugDecSep(j + 1, ja, n+1, ji, jai, sep, c, ca,
            sbn);
        gpH = gpH + res(2:3) * sb(1)/sumsb;
    end
    for i = 2:min(21-j,10)
        if sb(i) > 0
            sbn = sb; sbn(i) = sbn(i) - 1;
            res = JugDecSep(j+i, ja, n+1, ji, jai, sep, c, ca
                , sbn);
            gpH = gpH + res(2:3) * sb(i)/sumsb;
        end
    end
end
k = 22 - j; % Con cartas de valor k (o mayores) se pasa
if (k <= 10) && (ja==0) % La mano es dura; se pasa.
    gpH(2) = gpH(2) + sum(sb(k:10))/sumsb;
    if sep == 1 % Ahora comienza otra mano, con sep=0

```

```

        for i = k:10
            if sb(i) > 0
                sbn = sb; sbn(i) = sbn(i) - 1;
                for m = 1:10 % Dan un segundo naipe para
                    la otra mano.
                        if sbn(m) > 0
                            sbnn = sbn; sbnn(m) = sbnn(m) - 1;
                            res = JugDecSep(ji + m, jai, 2, ji
                                , jai, 0, c, ca, sbnn);
                            gpH = gpH + res(2:3) * sb(i)/sumsb
                                * sbn(m)/sum(sbn);
                        end
                    end
                end
            end
        end
    end
    if (k <= 10) && (ja==1) % La mano es blanda, se
        convierte en dura
        for i = k:10
            if sb(i) > 0
                sbn = sb; sbn(i) = sbn(i) - 1;
                res = JugDecSep(j+i-10, 0, n+1, ji, jai, sep,
                    c, ca, sbn);
                gpH = gpH + res(2:3) * sb(i)/sumsb;
            end
        end
    end
elseif j <= 10 % El jugador no puede pasarse, sabemos que la
    mano es dura
    gpS(1) = sum( m(17:(j-1)) ) + m(22); gpS(2) = sum( m((j
        +1):21) ) + m(1);
    if sb(1) > 0 % Si hay al menos un 'as' en el sabot. La
        mano pasa a blanda.
        sbn = sb; sbn(1) = sbn(1) - 1;
        res = JugDecSep(j+11, 1, n + 1, ji, jai, sep, c, ca,
            sbn); %j, ja, n, ji, jai, sep, c, ca, sb)
        gpH = gpH + res(2:3) * sb(1)/sum(sb);
    end
    for i = 2:10 % Recibe una nueva carta i con la que no se
        pasa.
        if sb(i) > 0
            sbn = sb; sbn(i) = sbn(i) - 1;
            res = JugDecSep(j+i, ja, n + 1, ji, jai, sep, c,
                ca, sbn); %j, ja, n, ji, jai, sep, c, ca, sb)
            gpH = gpH + res(2:3) * sb(i)/sum (sb);
        end
    end
end

```

```

        end
    end

elseif j > 21 && ja == 1
    j = j - 10; ja = 0;

elseif j > 21 && ja == 0
    gpS = [0,1]; gpH = [0,1];
end
if sep == 0
    esperanzaS = gpS(1) - gpS(2); esperanzaH = gpH(1) - gpH(2);
elseif sep == 1 % Si se planta, hay que evaluar lo que gana
    con la otra mano que queda
    decisionSegunda = JugadorDecide_S03(ji, jai, 1, ji, jai,
        0, c, ca, sb); %j, ja, n, ji, jai, sep, c, ca, sb)
    esperanzaS = gpS(1) - gpS(2) + decisionSegunda(1) -
        decisionSegunda(2);
    esperanzaH = gpH(1) - gpH(2);
end
if (esperanzaS >= esperanzaH)
    decision = [0, gpS(1), gpS(2), gpS(1), gpS(2), gpH(1),
        gpH(2), esperanzaS];
else
    decision = [1, gpH(1), gpH(2), gpS(1), gpS(2), gpH(1),
        gpH(2), esperanzaH];
end
end % Final de la función.

```

A.1.4. JugadorDobla(j,ja,n,c,ca,sb)

```

function [ salida ] = JugadorDobla(j,ja,n,c,ca,sb)
% Cálculo de la esperanza si el jugador no dobla.
jugNoDobla = JugadorDecide(j,ja,n,c,ca,sb);
% Inicializamos contadores si el jugador dobla.
jugDoblaG = 0; jugDoblaP = 0;
if j == 9 || j == 10 % Dobla con 9 o 10.
    if sb(1) > 0 % ¿Hay al menos un As?
        sbn = sb; sbn(1) = sbn(1) - 1;
        m = CrupierProb(c,ca,1,sbn);
        jugDoblaG = jugDoblaG + ( sum( m( 17 : j + 10) )
            + m(22) ) * sb(1)/sum(sb);
        jugDoblaP = jugDoblaP + (sum( m( j + 12 : 21) ) +
            m(1) ) * sb(1)/sum(sb);
    end
    for i = 2:10 % Cartas distintas al As.
        if sb(i) > 0

```

```

        sbn = sb; sbn(i) = sbn(i) - 1;
        m = CrupierProb(c,ca,1,sbn);
        jugDoblaG = jugDoblaG + ( sum( m( 17 : j + i - 1)
            ) + m(22) ) * sb(i)/sum(sb);
        jugDoblaP = jugDoblaP + (sum( m( j + i + 1 : 21)
            ) + m(1) ) * sb(i)/sum(sb);
    end
end
elseif j == 11 % Dobla con 11.
    for i = 1:10
        if sb(i) > 0
            sbn = sb; sbn(i) = sbn(i) - 1;
            m = CrupierProb(c,ca,1,sbn);
            jugDoblaG = jugDoblaG + ( sum( m( 17 : j + i - 1)
                ) + m(22) ) * sb(i)/sum(sb);
            jugDoblaP = jugDoblaP + ( sum( m( j + i + 1 : 21)
                ) + m(1) ) * sb(i)/sum(sb);
        end
    end
end
espJugDobla = (jugDoblaG - jugDoblaP) * 2;
espJugNoDobla = jugNoDobla(8);
% Toma de decisiones.
if espJugDobla >= espJugNoDobla
    salida(1) = 2; salida(2) = espJugDobla;
else
    salida(1) = 3;salida(2) = espJugNoDobla;
end
end % Final de la función.

```

A.1.5. JugadorAsegura(sb)

```

function [ seguro ] = JugadorAsegura( sb )
% sb: composición del sabot.
E = 2 * sb(10) / sum( sb ) - sum( sb(1:9) ) / sum(sb);
if E > 0
    seguro = 1; %Si debe asegurar devuelve '1'.
else
    seguro = 0; % En caso contrario devuelve '0'.
end
end

```

Bibliografía

- [1] EDUARD O. THORP (1962-1966) *Beat the dealer*. Vintage Books Edition.
- [2] STANFORD WONG (1975-1981) *Professional Blackjack*. Pi Yee Press, La Jolla, California, EE.UU.
- [3] PETER A. GRIFFIN (1979-1986) *The Theory of Blackjack*. Huntington Press, Las Vegas, Nevada, EE.UU.
- [4] ARNOLD SNYDER (1980-1983) *Blackbelt in Blackjack*. RGE, Berkeley, California, EE.UU.
- [5] KEN USTON (1985) *Million Dollars Blackjack*. Gambling Times Book. Los Angeles, California, EE.UU.

- [6] G. BRASSARD, P. BRATLEY (1997) *Fundamentos de Algoritmia*. Prentice Hall, Madrid.
- [7] (2001) BRIAN R. HUNT, RONALD L. LIPSMAN, JONATHAN M. ROSENBERG *A Guide to MATLAB for Beginners and Experienced Users*. Cambridge University Press, Edinburgh, Reino Unido.
- [8] THEODORE P. HILL El problema de la detención óptima. *Investigación y Ciencia*, febrero 2011, Pgs. 80-87.

1. Introduction

Blackjack is a very popular game offered by most casinos in the world. It is played with one, or several, French decks. Each card is worth its value in points, except the Ace that can be worth 1 or 11 and the figures, whose value is ten. The player should approach as closely as possible to 21 without going over. The player wins if he gets more points than the dealer without going over.



Figure 1: Usual Blackjack table

2. A mathematically advantageous game?

Unlike others games offered by the casinos, Blackjack was not explicitly conceived by calculating odds to give the dealer an advantage over the player, although from the beginning it was supposed to be the 'house' who won in the long run.

In the sixties of the twentieth century, several mathematicians from around the University of California began to study the odds of winning in Blackjack. Two were the initial questions that were asked: Is it possible to win in Blackjack? If possible, how much can be earned?

The answer to the first is 'yes': Blackjack is an advantageous game for the player if he makes the right decisions during the game. As for the second question, the results were somewhat discouraging from the start because the profit margin is, at best, very meager, at least with the classical methods to address this challenge.

3. Classical methods

There are three skill levels for someone who wants to win in Blackjack: the basic strategy (BS), the card count and the variation of the bets in combination with the

two previous options. The BS only refers to the player's score and the dealer's card.

	2	3	4	5	6	7	8	9	T	A
A,2	H	H	H	H	H	H	H	H	H	H
A,3	H	H	H	H	H	H	H	H	H	H
A,4	H	H	H	H	H	H	H	H	H	H
A,5	H	H	H	H	H	H	H	H	H	H
A,6	H	H	H	H	H	H	H	H	H	H
A,7	S	S	S	S	S	S	S	H	H	H
A,8	S	S	S	S	S	S	S	S	S	S
A,9	S	S	S	S	S	S	S	S	S	S

Table 1: Basic strategy for soft hands. H, hit. S, stand.

	2	3	4	5	6	7	8	9	T	A
4	H	H	H	H	H	H	H	H	H	H
5	H	H	H	H	H	H	H	H	H	H
6	H	H	H	H	H	H	H	H	H	H
7	H	H	H	H	H	H	H	H	H	H
8	H	H	H	H	H	H	H	H	H	H
9	H	D	D	D	D	H	H	H	H	H
10	D	D	D	D	D	D	D	H	H	H
11	D	D	D	D	D	D	D	H	H	H
12	H	H	S	S	S	S	H	H	H	H
13	S	S	S	S	S	S	H	H	H	H
14	S	S	S	S	S	S	H	H	H	H
15	S	S	S	S	S	S	H	H	H	H
16	S	S	S	S	S	S	H	H	H	H
17	S	S	S	S	S	S	S	S	S	S
18	S	S	S	S	S	S	S	S	S	S
19	S	S	S	S	S	S	S	S	S	S
20	S	S	S	S	S	S	S	S	S	S
21	S	S	S	S	S	S	S	S	S	S

Table 2: Basic strategy for hard hands, six decks and European standards. The top row indicates the points the dealer and the column on the left the points of the player. H, hit. S, stand. D, double.

The card count modifies the BS according to the composition of the sabot. For a player to memorize the indexes, they are rounded to whole numbers. In classical methods, these indices are calculated by simulations of thousands of items.

A	2	3	4	5	6	7	8	9	T
-1	1	1	1	1	1	0	0	0	-1

Table 3: A counting system of one level.

Finally, the modification of bets calculates if the cards remaining in the sabot are favorable for the player before each hand

begins, with which he bets more or less according to his advantage.

In this work we have set out to reach beyond these three levels of excellence in Blackjack, taking the calculation of probabilities to its ultimate consequences.

4. A new system

In order for a Blackjack player to be able to make the best decision at any time without depending on previously established rules, we have programmed a set of functions that calculate their probabilities at each moment of the game.

In this way, we have replaced the classic methods of simulations, later studied by statistical techniques, by a more precise determination.

We have used recursion to program these functions.

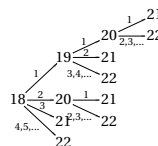


Figure 2: Tree with the possibilities of the player when he has 18 points.

References

- [1] EDUARD O. THORP. *Beat the dealer*. Vintage Books Edition, 1962-1966.
- [2] PETER A. GRIFFIN. *The Theory of Blackjack*. Huntington Press, Las Vegas, Nevada, USA, 1979-1986.
- [3] ARNOLD SNYDER. *Blackbelt in Blackjack*. RGE, Berkeley, California, USA, 1980-1983.
- [4] STANFORD WONG. *Professional Blackjack*. Pi Yee Press, La Jolla, California, USA, 1975-1981.
- [5] KEN USTON. *Million Dollars Blackjack*. Gambling Times Book. Los Angeles, California, USA, 1985.
- [6] G. BRASSARD, P. BRATLEY. *Fundamentos de Algoritmia*. Prentice Hall, Madrid, 1997